

目录

目录.....	1
1 引言	3
2 相关工作	4
3 数据集与数据预处理	5
3.1 数据来源与规模	5
3.2 字段说明	6
3.3 数据清洗与评分矩阵构建	6
3.4 训练/测试集划分	7
3.5 探索性数据分析	7
4 系统需求分析	8
5 系统设计	9
5.1 总体架构	9
5.2 数据流	10
5.3 数据库设计	11
6 推荐算法设计	12
6.1 基线模型	12
6.2 基于用户的协同过滤 (UserCF)	12
6.3 基于物品的协同过滤 (ItemCF)	12
6.4 基于偏置残差的 SVD 矩阵分解	13
6.5 元数据混合模型与神经协同过滤	13
6.6 推荐理由与相似/热门推荐	13
7 系统实现	13
7.1 系统欢迎页与角色入口	14
7.2 用户注册、登录与偏好选择	14
7.3 用户端电影库与电影详情	15
7.4 个性化推荐与相似/热门推荐	18
7.5 用户评分、收藏与画像分析	21
7.6 数据可视化	25
7.7 管理员后台功能实现	25
8 实验设计与模型评估	30
8.1 评估协议与指标	30
8.2 实验数	31
8.3 评估结果	31
8.4 相关性阈值敏感性	32
8.5 超参数敏感性	33
8.6 NeuralCF 训练过程可视化	33
9 结果分析	34
9.1 定量结果分析	34
9.2 错误案例与原因分析	34
9.3 覆盖率与多样性	35
9.4 冷启动与定性结果	35
9.5 局限性	36

10 项目测试与运行说明 36

 10.1 运行环境与依赖 36

 10.2 数据与目录 36

 10.3 运行步骤 36

 10.4 可复现性 37

11 团队分工 37

 11.1 分工说明 37

 11.2 项目反思 38

12 Git 版本控制与项目管理 38

13 结论与展望 39

参考文献 40

附录 41

 附录 A 实验记录表（含未达预期的尝试） 41

 附录 B 可复现性与实验日志 41

摘要

在线影视平台内容规模的持续膨胀使用户面临严重的信息过载，个性化推荐由此成为提升内容触达效率的关键手段。本项目以 GroupLens 发布的 MovieLens 100K 数据集（943 名用户、1,682 部电影、100,000 条 1–5 分评分，矩阵稀疏度约 93.7%）为基础，设计并实现了名为 FilmTrace 的智能电影推荐系统。在算法层面，项目围绕评分预测与 Top-N 排序两类任务，实现并统一对比了 11 种模型：包括全局/用户/物品均值、正则化偏置、随机评分与最受欢迎在内的 6 类基线，基于用户与基于物品的协同过滤（UserCF、ItemCF），基于偏置残差的 TruncatedSVD 矩阵分解，融合电影元数据的混合回归模型，以及基于 PyTorch 的神经协同过滤（NeuralCF）。评估采用按用户时间序列划分的训练/测试集（固定随机种子 42），以 RMSE、MAE 衡量评分预测精度，以 Precision@10、Recall@10、HitRate@10、NDCG@10 与目录覆盖率衡量推荐列表质量。实验结果显示，偏置残差 SVD 取得最优的 RMSE（0.9826）与 MAE（0.7763），优于 NeuralCF（RMSE 1.0080）等其余模型；而在全目录离线 Top-N 协议下，流行度类基线在排序指标上反超个性化模型，报告结合评估协议与数据稀疏性对该现象进行了分析。在系统层面，项目基于 Streamlit 与 SQLite 构建了涵盖用户端与管理员端的完整应用，集成账号认证、偏好选择、个性化推荐、评分收藏、用户画像、数据可视化与后台管理等功能，并配套单元测试与持续集成，具备良好的工程完整性、可复现性与实用价值。

关键词：电影推荐系统；MovieLens 100K；协同过滤；矩阵分解；Streamlit；数据可视化

1 引言

在流媒体与影视点播高度普及的今天，平台可供选择的影片数量动辄数以万计，用户在有限时间内难以从海量内容中定位真正契合自身口味的影片。这种供给远大于个体消化能力的“信息过载”现象，既降低了用户的观影体验，也削弱了平台的内容分发效率。推荐系统正是为缓解这一矛盾而生：它通过建模用户的历史行为与内容特征，主动为每位用户筛选并排序其潜在感兴趣的内容，已成为现代内容平台不可或缺的核心组件。

电影是推荐系统研究中最具代表性的对象之一。一方面，电影具备类型、年代、主创等丰富的结构化属性，便于建模；另一方面，MovieLens 系列数据集长期作为协同过

滤与矩阵分解算法的公开基准，使研究结果可与既有工作横向比较。基于此，本项目选择 MovieLens 100K 作为数据基础，围绕“评分预测”和“Top-N 排序推荐”两个核心问题展开。前者指在用户的稀疏历史评分上预测其对未观看电影的评分，属于回归问题；后者指从用户未评分的影片中选出并排序最值得推荐的若干部，属于排序问题。

从应用视角看，本项目需要解决三类具体痛点：新注册用户缺乏历史评分而难以启动推荐（冷启动）；普通用户缺乏对平台整体评分分布与自身观影偏好的直观认识；平台运营方缺少对电影库、用户数据与算法参数的集中管理与评估手段。为此，FilmTrace 通过注册时的偏好类型选择缓解冷启动，通过用户画像与数据可视化提升推荐的可解释性，并通过管理员后台实现对电影、用户、评分、算法与评估的统一管控。

本项目的目标可概括为三点：其一，在统一的数据划分与评估协议下，实现并对比多类推荐算法，厘清不同方法在评分预测与排序任务上的优劣；其二，建立涵盖误差类与排序类指标的多维评估体系，并辅以超参数敏感性、冷启动与误差分层等诊断分析；其三，将上述算法落地为一套可演示、可复现的 Web 推荐系统，作为课程项目的综合实践成果。

2 相关工作

推荐系统的研究可大致归为协同过滤、基于内容的方法、混合方法以及近年来兴起的深度学习方法。协同过滤（Collaborative Filtering）是其中应用最为广泛的一类，其核心假设是“行为相似的用户在未知物品上的偏好也相近”。基于邻域的协同过滤按相似度计算对象的不同，又分为基于用户与基于物品两种。Sarwar 等人较早系统地研究了基于物品的协同过滤，指出在大规模场景下物品相似度比用户相似度更稳定、更易离线预计算，从而兼顾推荐质量与可扩展性[2]。

在评分预测任务上，矩阵分解（Matrix Factorization）凭借 Netflix Prize 的推动成为主流。Koren 等人对隐因子模型进行了系统总结，强调在隐因子之外显式引入全局均值与用户/物品偏置项，能够有效刻画评分的系统性偏差并显著提升预测精度[3]。本项目实现的 SVD 模型正是沿用这一思想，采用“先用偏置基线拟合整体水平、再对评分残差做低秩分解”的两阶段策略，以更好地适应高度稀疏的评分矩阵。

推荐系统的评估方法本身也是研究重点。Herlocker 等人指出，单一的预测误差指标无法全面反映推荐列表的排序质量与用户体验，需结合查准率、查全率、归一化折损累

积增益（NDCG）及覆盖率、多样性等多维指标综合衡量[4]。Cremonesi 等人进一步揭示，在离线 Top-N 评估中，未被评分的物品并非真实负样本，全目录排序评估天然有利于流行度基线，使其在 Precision/Recall 上表现突出[5]——这一结论对理解本项目第 9 章的实验现象至关重要。

为缓解数据稀疏与冷启动问题，混合推荐将协同过滤与基于内容/元数据的方法相结合。Burke 对混合推荐进行了系统综述，归纳了加权、切换、特征组合等多种融合范式[6]。在深度学习方向，He 等人提出的神经协同过滤（Neural Collaborative Filtering）以嵌入向量与多层感知机替代传统的内积运算，建模用户与物品之间的非线性交互，拓展了协同过滤的表达能力[7]。本项目在传统基线、UserCF、ItemCF、SVD 与混合模型之外，亦实现了基于 PyTorch 的 NeuralCF，并将其纳入统一评估，以考察深度方法在中小规模显式评分数据上的实际表现。MovieLens 数据集由 GroupLens 长期维护，是上述各类方法最常用的公开评测基准之一[1]。

近三年来，推荐系统研究持续向深度化、多样化方向演进。Gheewala 等人于 2025 年对深度学习推荐模型进行了系统综述，覆盖评分预测与 Top-N 排序两类任务，梳理了从传统到现代再到混合架构的演进脉络及其数据集与特征分析[13]。Chen 等人系统总结了深度强化学习在推荐系统中的应用与新视角，强调将推荐建模为序贯决策问题以优化长期收益[14]。Liu 等人针对隐私保护需求，对基于深度学习的联邦推荐系统及其聚合策略进行了实验对比[15]；Talha 等人则面向新闻推荐场景综述了深度学习方法的挑战与未来趋势[16]。这些最新进展表明，图神经网络、强化学习、联邦学习与大语言模型正成为推荐系统的研究前沿；本项目聚焦经典协同过滤、矩阵分解与基础神经协同过滤的实现与对比，是理解上述前沿方法的基础。

3 数据集与数据预处理

3.1 数据来源与规模

本项目采用 GroupLens 研究组发布的 MovieLens 100K 数据集，它是推荐系统领域使用最为广泛的公开基准之一。数据集主要包含三个文件：u.data 记录评分三元组（用户、电影、评分）及其 Unix 时间戳；u.item 记录电影元数据，包括片名、上映日期、IMDb 链接以及 19 个类型的二值标记；u.genre 给出类型名称列表。数据集的整体规模

与稀疏程度如表 1 所示。

表 1 MovieLens 100K 数据集基本信息

统计项	数值	备注
用户数	943	每位用户至少评分 20 部电影
电影数	1,682	覆盖 19 个类型标记
评分总数	100,000	评分为 1-5 的整数
评分矩阵规模	$943 \times 1,682$	理论单元约 158.6 万
实际填充率	$\approx 6.3\%$	矩阵稀疏度约 93.7%
附加信息	时间戳	用于按时间划分训练/测试集

3.2 字段说明

项目实际使用的关键字段及其用途如表 2 所示。

表 2 主要字段说明

字段	来源文件	用途
user_id / movie_id	u.data / u.item	用户与电影标识，构建评分矩阵
rating	u.data	1-5 评分，作为训练与评估的标签
timestamp	u.data	评分时间，用于按用户时序划分数据
title / release_year	u.item	片名与上映年份，用于展示与年代分析
genres (19 类)	u.item / u.genre	类型标记，用于类型统计与混合模型特征
review	系统 ratings 表	用户文字评价，运行时写入 SQLite

3.3 数据清洗与评分矩阵构建

原始数据虽经 GroupLens 整理，仍需针对建模需求进一步处理。清洗工作由 src/data_cleaning.py 与 src/preprocessing.py 完成，主要环节见表 3。处理完成后，以 (user_id, movie_id) 为索引构建用户-电影评分矩阵，作为协同过滤与矩阵分解的统一输入。

表 3 数据预处理流程

环节	具体处理
类型转换	将 user_id、movie_id、rating、timestamp 转为数值类型，非法值置空后剔除
完整性校验	校验评分必备字段是否齐全，剔除关键字段缺失的记录
去重	以 (user_id, movie_id) 为主键，保证同一用户对同一电影仅有一条评分
年份解析	用正则从片名中提取上映年份，生成 release_year 字段

环节	具体处理
类型解析	将 19 列类型二值标记转换为 genres 列表，便于展示与特征工程
取值核验	确认评分严格落于 1–5 区间，异常值按规则处理
海报匹配	src/posters.py 规范化片名后匹配本地海报，未命中时使用统一占位图

需特别说明：推荐系统场景一般不涉及图像/文本类的数据增强（data augmentation），因此本项目未采用数据增强策略，而是通过偏置正则化与偏置残差分解等建模手段应对评分矩阵的高度稀疏问题。

3.4 训练/测试集划分

为避免“以未来数据预测过去”的信息泄漏，主实验采用按用户时间序列划分（per-user temporal split）：对每位用户的评分按时间戳排序，较早的部分作为训练集、较晚的部分作为测试集，测试比例为 0.2，同时保证每位被评估用户都拥有训练历史。所有涉及随机性的步骤统一固定随机种子 `RANDOM_STATE=42`。作为对照，项目亦实现了全局时间划分与随机划分（src/preprocessing.py），仅用于稳健性参考。

3.5 探索性数据分析

建模前对数据进行了探索性分析。图 1 给出评分分布，图 2 给出各类型电影数量分布。

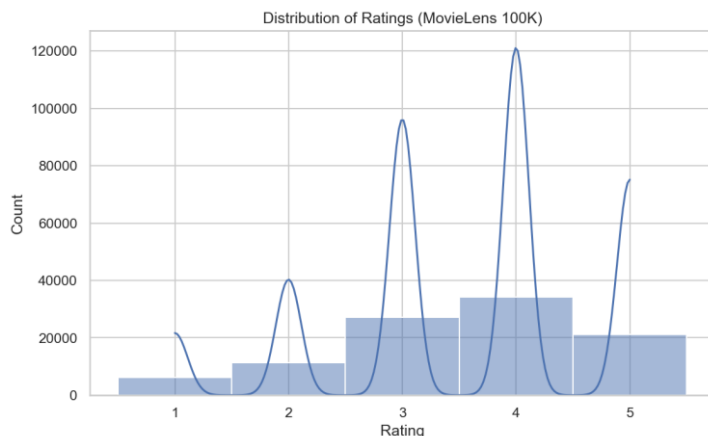


图 1 MovieLens 100K 评分分布直方图

由图 1 可见，评分集中于 3–4 分、整体呈左偏分布，表明用户更倾向于给出中性偏正面的评价。这一特征使得用户均值、全局均值等简单基线在 RMSE/MAE 上也能取得不俗成绩，是后文比较各模型差距时的重要背景。

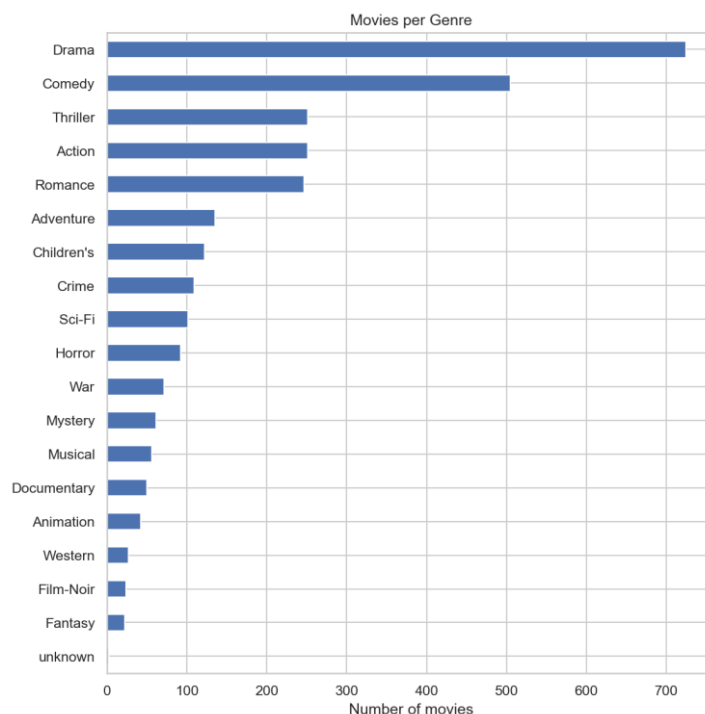


图 2 电影类型数量分布

由图 2 可见，Drama、Comedy、Action 等类型电影数量明显居前，类型分布存在显著长尾，这为后文讨论“个性化精度与目录覆盖率之间的权衡”提供了数据层面的解释。

4 系统需求分析

在算法之外，本项目需将推荐能力封装为可运行、可演示的系统。为此，从用户端、管理员端、推荐、可视化、评估、数据管理与可复现性等维度梳理功能需求，归纳如表 4。需求整体围绕两条主线组织：用户端以“浏览—推荐—反馈”为主线，强调个性化与可解释；管理员端以“管理—配置—评估”为主线，强调对数据与算法的可控性。

表 4 功能模块需求

类别	模块	需求说明
用户端	账号与偏好	支持注册、登录，并在注册时选择偏好类型以支持冷启动推荐
用户端	浏览与推荐	提供电影库、电影详情、热门电影、个性化推荐与相似电影推荐
用户端	反馈与画像	支持评分、文字评价、收藏，并据行为生成用户画像与数据可视化
管理员端	数据管理	支持对电影、用户、评分记录的查询与管理
管理员端	算法与评估	支持推荐算法配置、模型评估与系统统计分析
数据管理	持久化	对账号、收藏、偏好、画像、评价等数据进行持久化存储

类别	模块	需求说明
工程	可复现性	固定随机种子，提供依赖文件、运行说明、单元测试与版本管理

5 系统设计

5.1 总体架构

FilmTrace 采用“单体 Streamlit 应用 + SQLite/本地文件存储”的架构，不引入独立后端服务，以契合课程项目的规模与可复现要求。系统总体架构如图 3 所示，自下而上可划分为六个层次。

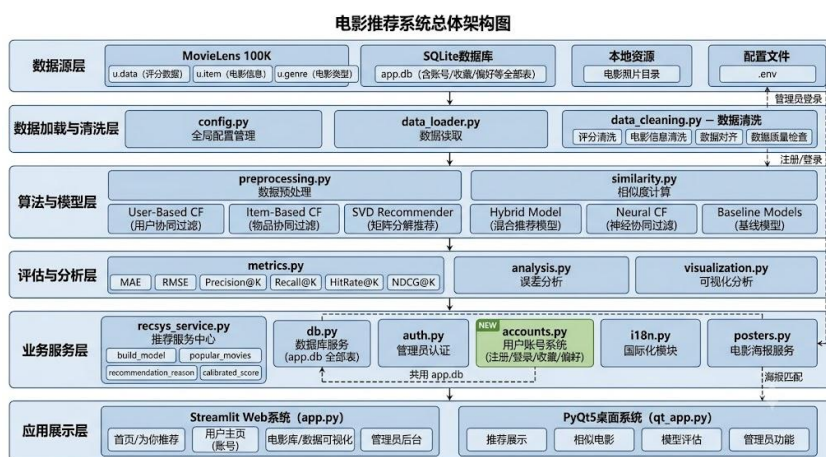


图 3 电影推荐系统总体架构图

各层职责如下：

- 数据源层：MovieLens 100K 原始文件（u.data / u.item / u.genre）与本地电影海报资源；
- 数据加载与清洗层：由 data_loader、data_cleaning、preprocessing、posters 完成数据读取、清洗、训练/测试划分、评分矩阵构建与海报匹配；
- 算法/模型层：承载 11 种推荐模型（基线、UserCF、ItemCF、SVD、Hybrid、NeuralCF）与在线推荐服务 recsys_service；
- 评估与分析层：实现 RMSE/MAE 与 Precision@K/Recall@K/HitRate@K/NDCG@K/覆盖率等指标（metrics）及诊断分析（analysis）；
- 业务服务层：组织账号认证、偏好、收藏、画像、审计等业务逻辑，并通过 db 模块读写 SQLite；

- 应用展示层：基于 Streamlit 渲染用户端与管理员端界面，完成页面导航、表单交互与图表展示。

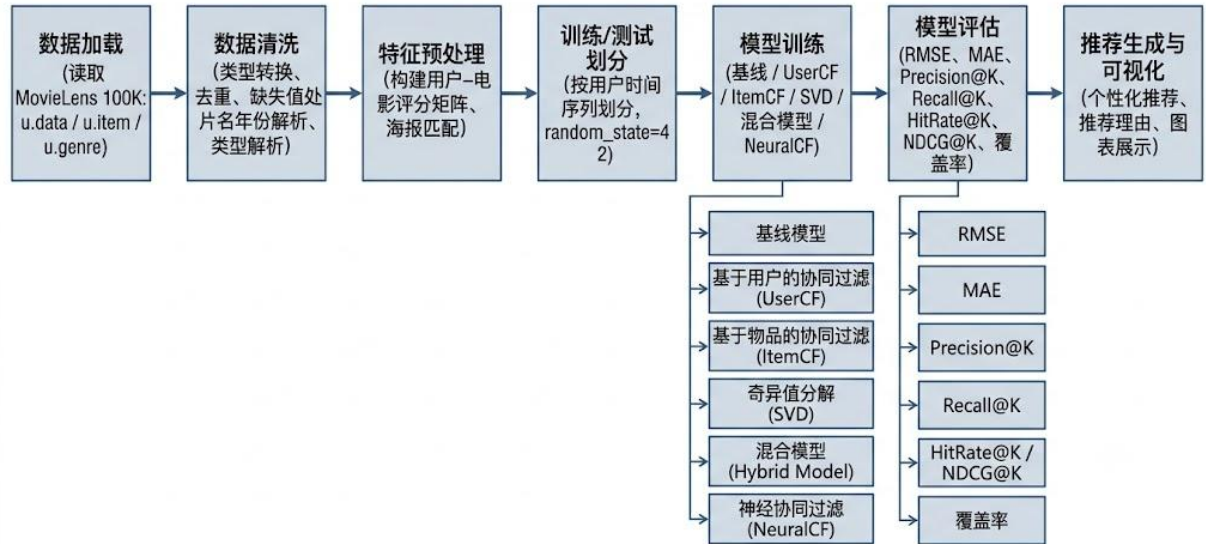


图 4 推荐系统方法处理流程图

图 4 给出了本项目从原始数据到推荐结果的完整方法流程。系统首先加载 MovieLens 100K 的 u.data、u.item、u.genre 文件，经数据清洗（类型转换、去重、缺失值与年份/类型解析）与特征预处理后构建用户-电影评分矩阵；随后按用户时间序列将数据划分为训练集与测试集（固定随机种子 42），以避免信息泄漏。在训练阶段，分别训练基线模型、UserCF、ItemCF、基于偏置残差的 SVD、元数据混合模型与基于 PyTorch 的 NeuralCF；在评估阶段，统一使用 RMSE、MAE 与 Precision@K、Recall@K、HitRate@K、NDCG@K 及覆盖率等指标对各模型进行对比。最终，系统基于评估后选定的算法生成个性化推荐结果并附带推荐理由，同时通过数据可视化模块对评分分布、类型偏好与模型表现进行展示。该流程清晰地体现了“数据—模型—评估—推荐”的完整闭环。

5.2 数据流

系统主数据流为：原始文件经数据加载与清洗层形成评分矩阵与电影元数据；用户在展示层产生的注册、偏好、评分、收藏等行为经业务服务层写入 SQLite；算法层结合评分矩阵、元数据与用户实时行为计算推荐，再经展示层以海报卡片呈现；评估层在离线阶段统一评估各模型，结果可在管理员端“模型评估”页查看。

5.3 数据库设计

系统以 SQLite 数据库文件 app.db 作为物理存储，读写逻辑统一封装于 src/db.py，是真实的关系型数据库而非仅逻辑模型。数据库共 8 张表，结构概览如表 5，主要实体关系见图 5 的 ER 图。

表 5SQLite 数据表结构概览

数据表	主键	主要字段与用途
movies	movie_id	title、data（元数据 JSON），电影库信息
users	user_id	n_ratings、avg_rating，MovieLens 用户评分统计
ratings	(user_id, movie_id)	rating、timestamp、review，评分与文字评价
accounts	account_id	username、email、password_hash、movielens_user_id，登录账号
favorites	(account_id, movie_id)	created_at，用户收藏记录
user_preferences	account_id	genres、seed_movie_ids，注册偏好类型与种子电影
user_profiles	account_id	top_genres、year_range、preferred_min_rating，用户画像
admin_audit_log	id	ts、operation、administrator、target，管理员操作审计

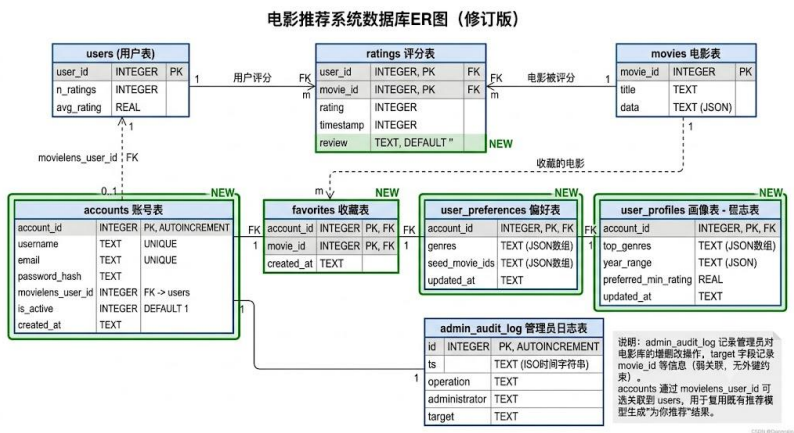


图 5 电影推荐系统数据库 ER 图

主要实体关系说明如下：

- accounts 为系统登录账号，通过 movielens_user_id 可选关联到 MovieLens 的 users，从而复用其历史评分生成推荐；
- ratings 作为 users 与 movies 之间多对多关系的连接实体，承载评分与评价；
- favorites、user_preferences、user_profiles 均以 account_id 关联 accounts，分别承载收藏、偏好与画像，体现“一个账号对应一份偏好/画像、多条收藏”的关系；

- `admin_audit_log` 独立记录管理员写操作，用于权限留痕与审计。其中 `movies`、`users`、`ratings` 在首次运行时由原始数据落库，其余表则在用户与管理交互过程中动态写入。

6 推荐算法设计

本项目共实现 11 种模型，分为传统方法与深度学习方法两类。传统方法包括 6 类基线、UserCF、ItemCF、SVD 与混合模型，它们通过解析解、邻域加权或矩阵分解直接求解，不依赖梯度迭代训练；深度学习方法为基于 PyTorch 的 NeuralCF。下面分别介绍各核心算法的思想、实现与适用性。

关于框架选型：项目主体算法属可解析或可分解求解的传统模型，无需 GPU 训练，使用 `scikit-learn` 与 `NumPy` 即可高效实现并保证可复现；对于需要端到端梯度训练的 NeuralCF，则按课程要求优先选用 PyTorch，因其动态计算图便于调试、自动求导接口简洁、生态成熟。这种“传统模型用 `sklearn`、深度模型用 PyTorch”的混合选型兼顾了效率、可复现与对深度方法的探索。

6.1 基线模型

6 类基线为各模型提供统一参照系：全局/用户/物品均值直接取相应子集评分的算术平均；正则化偏置基线（BiasBaseline）以带 L2 正则项的方式求解“全局均值 + 用户偏置 + 物品偏置”，敏感性分析确定正则化系数 `reg=1.0` 为最优；随机评分在训练集评分范围内均匀采样，作为质量下界；最受欢迎（Most Popular）按评分次数排序，作为流行度参照。

6.2 基于用户的协同过滤（UserCF）

UserCF 以“兴趣相近的用户偏好相近”为假设，在训练矩阵上计算用户两两相似度（余弦/皮尔逊），为目标用户选取相似度最高的 `K` 个近邻（`K_NEIGHBORS=20`），以近邻评分的相似度加权平均作为预测。其优点是直观、可解释；局限在于用户规模增大时相似度计算成本高，且稀疏数据下用户间共同评分稀少，相似度估计方差较大。

6.3 基于物品的协同过滤（ItemCF）

ItemCF 将相似度建立在电影之间，若两部电影被大量用户给出相近评分则视为相似。系统为目标用户已评分的电影寻找相似电影并加权汇总生成推荐。相比 UserCF，物品相似度通常更稳定、更便于离线预计算，适合电影数量相对固定的场景；其局限同样在于对评分稀少的冷门电影存在冷启动困难。

6.4 基于偏置残差的 SVD 矩阵分解

SVD 模型采用两阶段策略：先用正则化偏置基线拟合评分的整体水平，再对评分残差矩阵执行 TruncatedSVD 低秩分解（SVD_N_COMPONENTS=50），得到用户与电影的隐因子向量；最终预测为“偏置预测 + 用户隐向量与电影隐向量内积”，并裁剪至 [1,5]。相比直接分解原始稀疏矩阵，先扣除偏置再分解残差能降低稀疏性带来的不稳定，更充分地刻画用户与电影的潜在交互，因而在评分预测上取得本项目最优精度。

6.5 元数据混合模型与神经协同过滤

元数据混合模型（MetadataHybridRegressor）以用户/物品偏置、用户对各类型的历史均分、电影类型与年份等特征训练回归模型，对评分稀少或新上架电影更鲁棒，是系统应对冷启动的算法手段之一。神经协同过滤（NeuralCF，src/neural_cf.py）基于 PyTorch，将用户与电影分别映射为 32 维嵌入向量，拼接后送入隐藏层维度为 (64, 32) 的多层感知机（各层接 ReLU 与 Dropout），以 MSE 损失与 AdamW 优化器端到端训练 20 个 epoch，借非线性交互建模评分。其训练过程与评估结果详见第 8 章。

6.6 推荐理由与相似/热门推荐

为提升可解释性，在线推荐服务（src/recsys_service.py）会依据所用算法生成自然语言推荐理由：采用 UserCF 时强调“与你评分行为相似的用户给出高分”，采用 SVD 时强调“潜在因子显示你对该片偏好较高”，基于物品/类型相似度时强调“与你评分过的某类型影片相似”；对缺乏历史的新用户，则结合注册偏好类型与热门/高分电影给出推荐。相似电影推荐基于电影类型相似度实现，热门电影排序与“最受欢迎”基线在逻辑上一致，二者共同构成系统中基于内容与流行度的补充推荐路径。

7 系统实现

本章以实际运行截图为依据，自用户端至管理员端逐一展示 FilmTrace 的功能实现。对每个模块，均说明其功能定位、所用数据与交互方式，并阐明其在整个推荐系统

中的作用。系统整体采用深色影院风格，视觉统一。

7.1 系统欢迎页与角色入口

图 6 所示为 FilmTrace 欢迎页，是用户进入系统的统一入口。页面突出展示系统名称与“智能电影推荐平台”定位，并以数据卡片呈现 MovieLens 100K 的核心规模（943 名用户、1,682 部电影、100,000 条评分），同时以标签形式标注 UserCF、ItemCF、SVD、NeuralCF 等核心算法，使访问者在进入前即可了解系统的数据规模与技术路线。页面下方分别提供用户端与管理端入口，实现两类角色的分流。

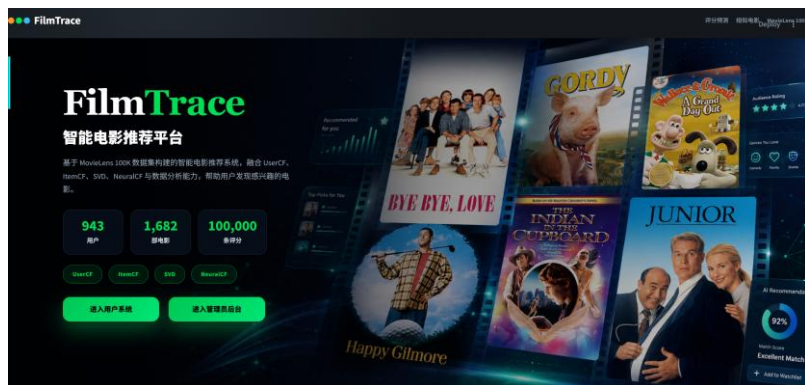


图 6 FilmTrace 系统欢迎页

7.2 用户注册、登录与偏好选择

图 7 所示为用户登录界面。用户凭用户名与密码登录，凭据由 `src/accounts.py` 校验，账号数据存于 SQLite 的 `accounts` 表，密码以哈希形式保存。登录后系统通过会话状态记录账号与角色，并据角色渲染对应的用户端或管理员端导航，实现访问隔离。



图 7 用户登录界面

图 8 所示为用户注册与偏好选择界面。新用户注册时除填写账号信息外，还需从类型列表中勾选感兴趣的电影类型。该偏好写入 `user_preferences` 表的 `genres` 字段，并

在用户尚无评分历史时作为生成初始推荐的主要依据，是系统应对新用户冷启动的关键设计。

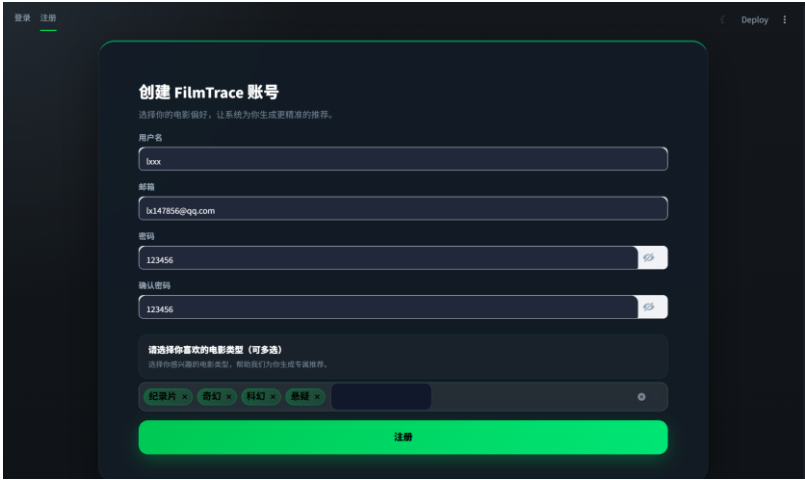


图 8 用户注册与偏好选择界面

结合第 5 章可知，本页提交的数据最终落入 accounts 与 user_preferences 两张表，二者以 account_id 关联。

7.3 用户端电影库与电影详情

图 9 所示为用户登录后的首页，以导航形式整合“为你推荐”“电影库”“我的收藏”“评分记录”“我的画像”等主要入口，是用户完成个性化推荐工作流的起点，可由此快速跳转至浏览或推荐。

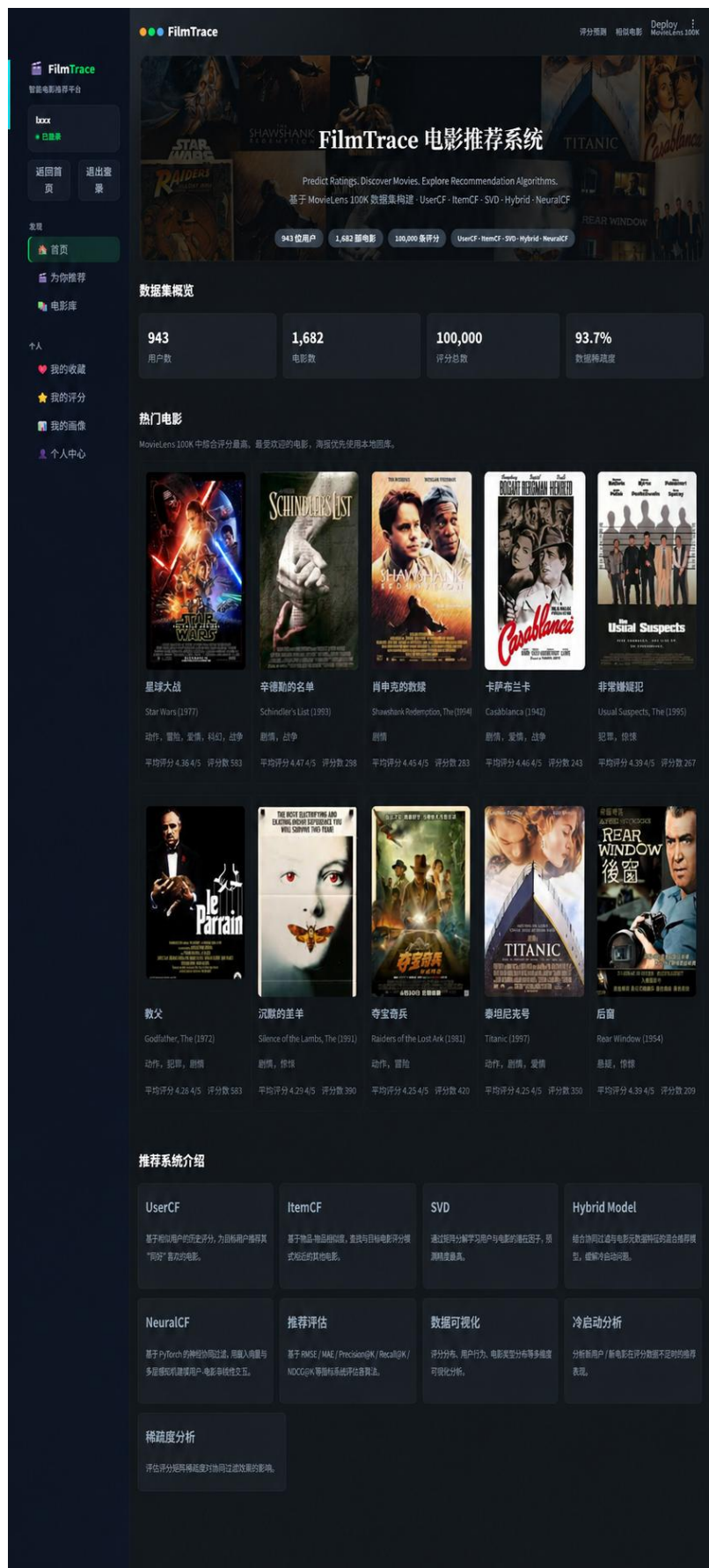


图 9 用户端首页界面

图 10 所示为用户端电影库，以海报卡片批量展示数据集中的电影。每张卡片包含由 src/posters.py 从本地匹配的海报、片名、上映年份与类型标签，并支持按类型、年代检索筛选。用户可在此浏览全部电影并进行评分、收藏，是浏览数据集内容的主要入口。

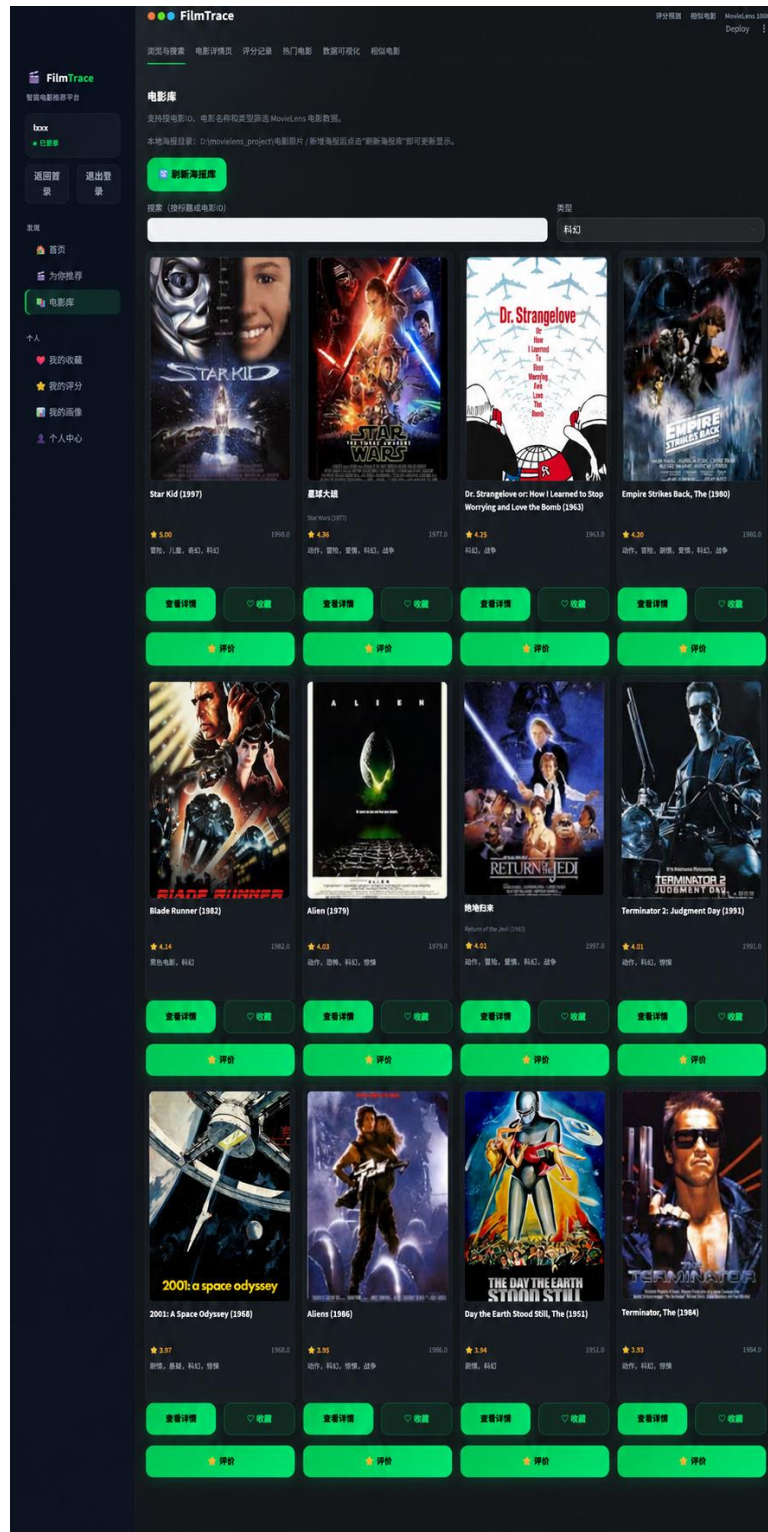


图 10 用户端电影库界面

图 11 所示为单部电影的详情页，呈现片名、上映年份、类型标签、本地海报及（如有）IMDb 链接等元数据，为用户提供比列表更完整的信息，辅助其做出评分、收藏或观看决策。

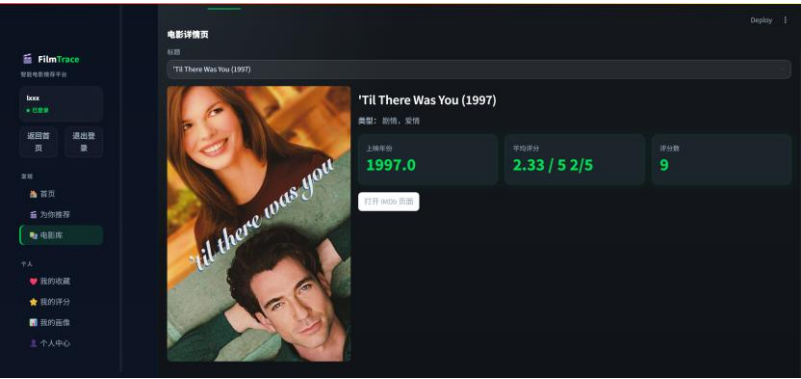


图 11 电影详情展示界面

图 12 所示为从详情页跳转的 IMDb 信息页，作为本地元数据的外部补充，为用户提供该片在 IMDb 上的参考信息。

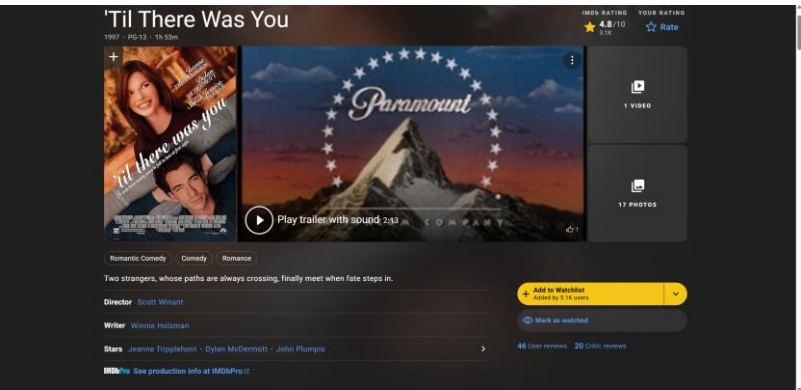


图 12 电影 IMDb 信息展示界面

7.4 个性化推荐与相似/热门推荐

图 13 所示为“为你推荐”页的个性化推荐结果。系统结合用户历史评分与注册偏好，调用第 6 章所述协同过滤与偏置/SVD 模型计算候选电影的预测评分，以海报卡片展示推荐列表，并标注预测评分与推荐理由。该页是系统核心推荐能力的直接呈现。

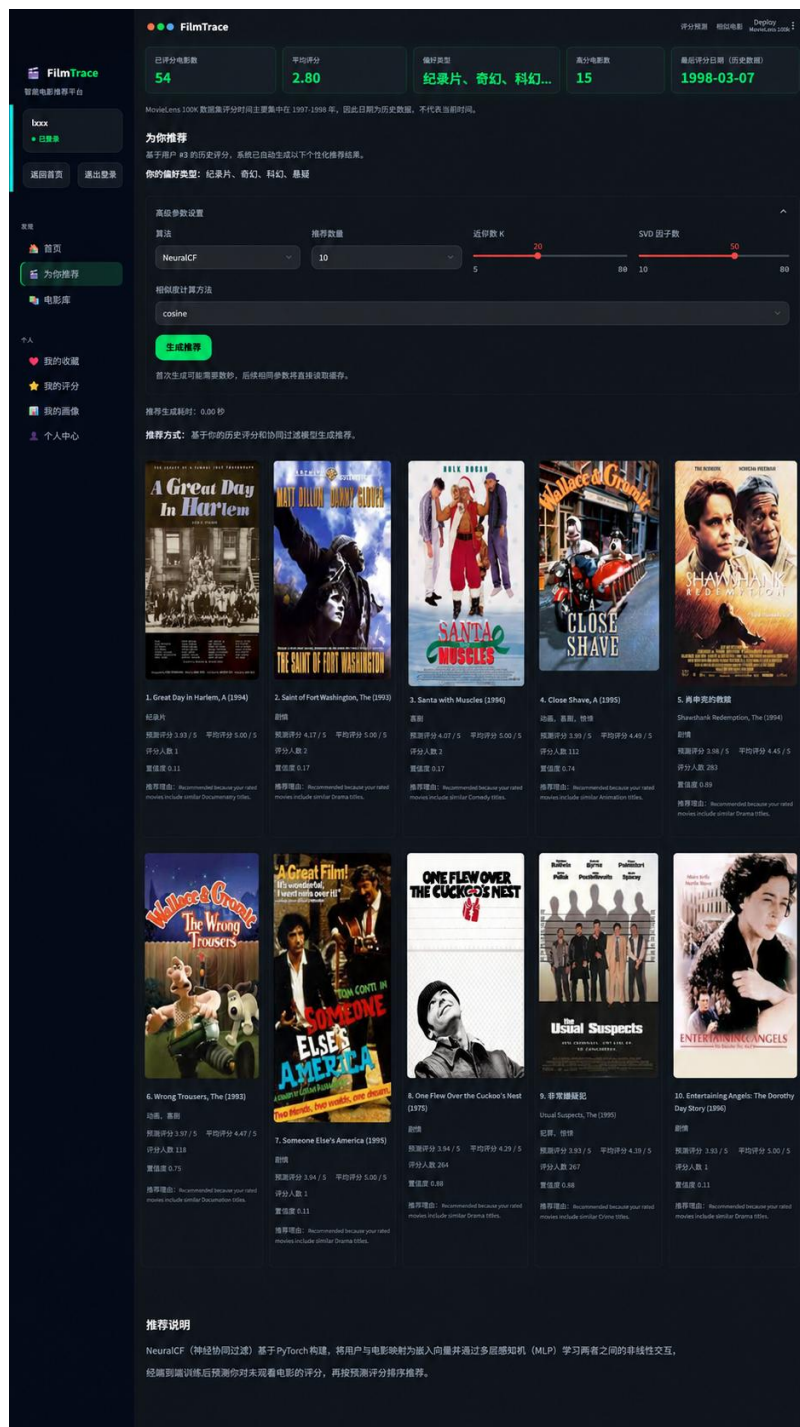


图 13 个性化推荐结果展示界面

卡片所示预测评分区间基于离线残差分位数给出，为近似参考而非严格置信区间。

图 14 所示为相似电影推荐页。系统基于电影类型相似度，为用户当前浏览的电影推荐风格相近的其他电影，对应 recsys_service 中基于类型相似度的逻辑，为用户提供“继续探索”的延伸路径。

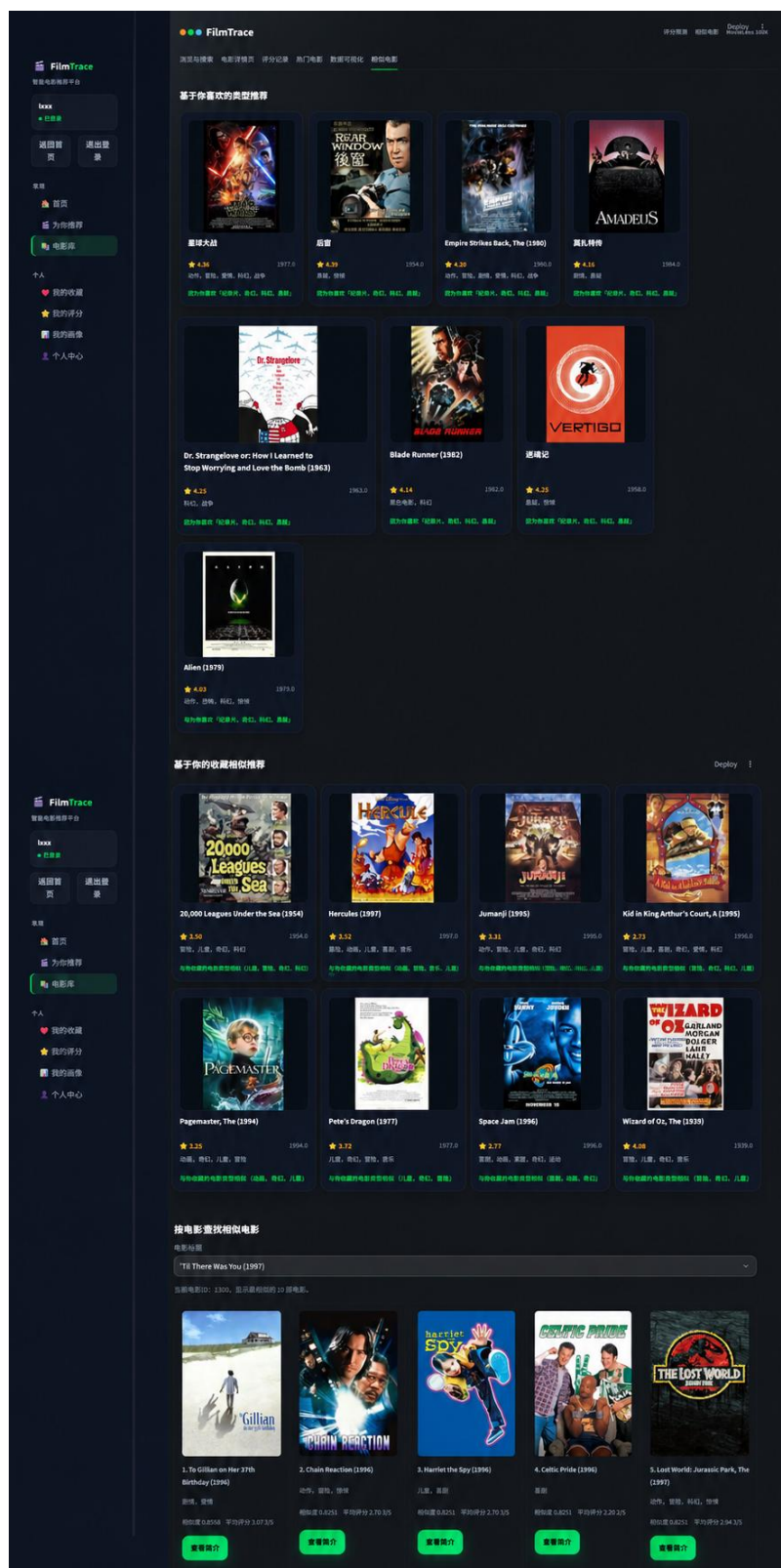


图 15 所示为电影库“热门电影”视图，按被评分次数（流行度）排序展示。该视图与第 6 章“最受欢迎”基线逻辑一致，也是第 9 章讨论“流行度基线在 Top-N 上反超个性化模型”现象的直观印证。

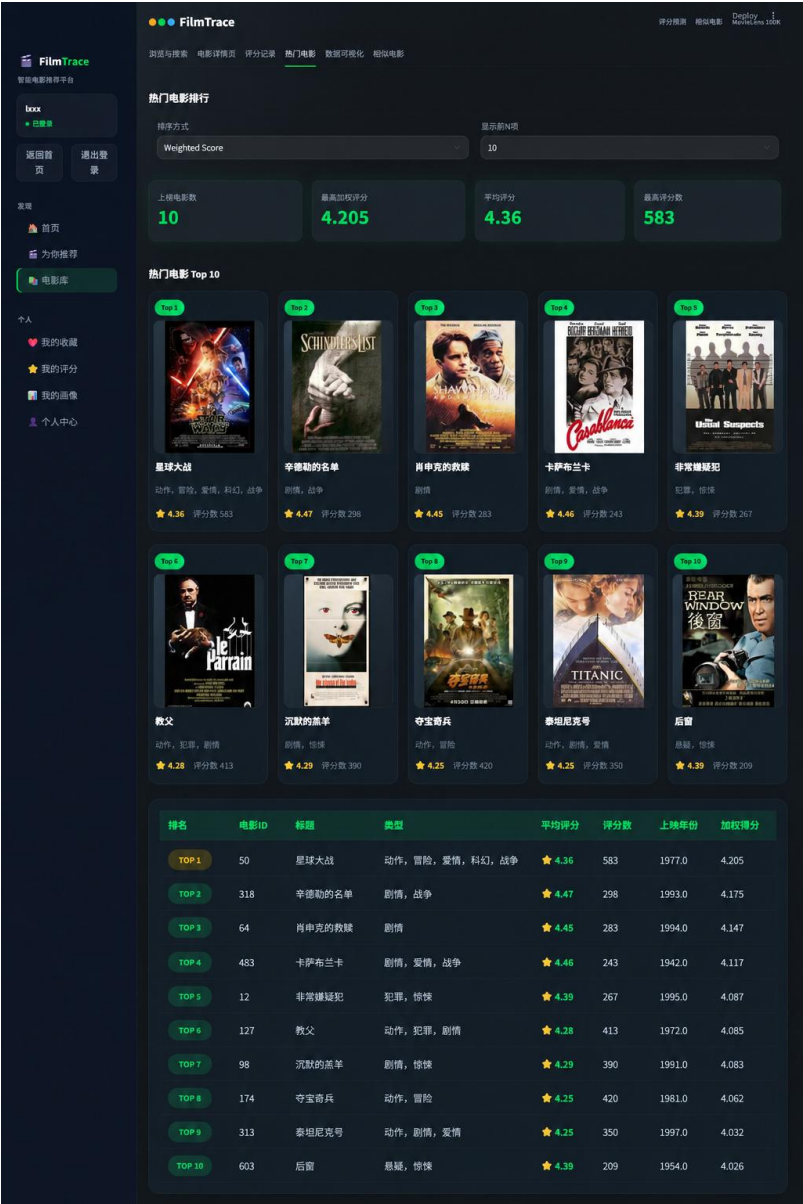


图 15 电影库热门电影界面

7.5 用户评分、收藏与画像分析

图 16 所示为评分记录与偏好分析界面，汇总用户历史评分并提供评分次数、平均评分、最高评分电影等统计。数据来源于 ratings 表，是个性化推荐的重要输入，也直观反映用户的观影偏好。

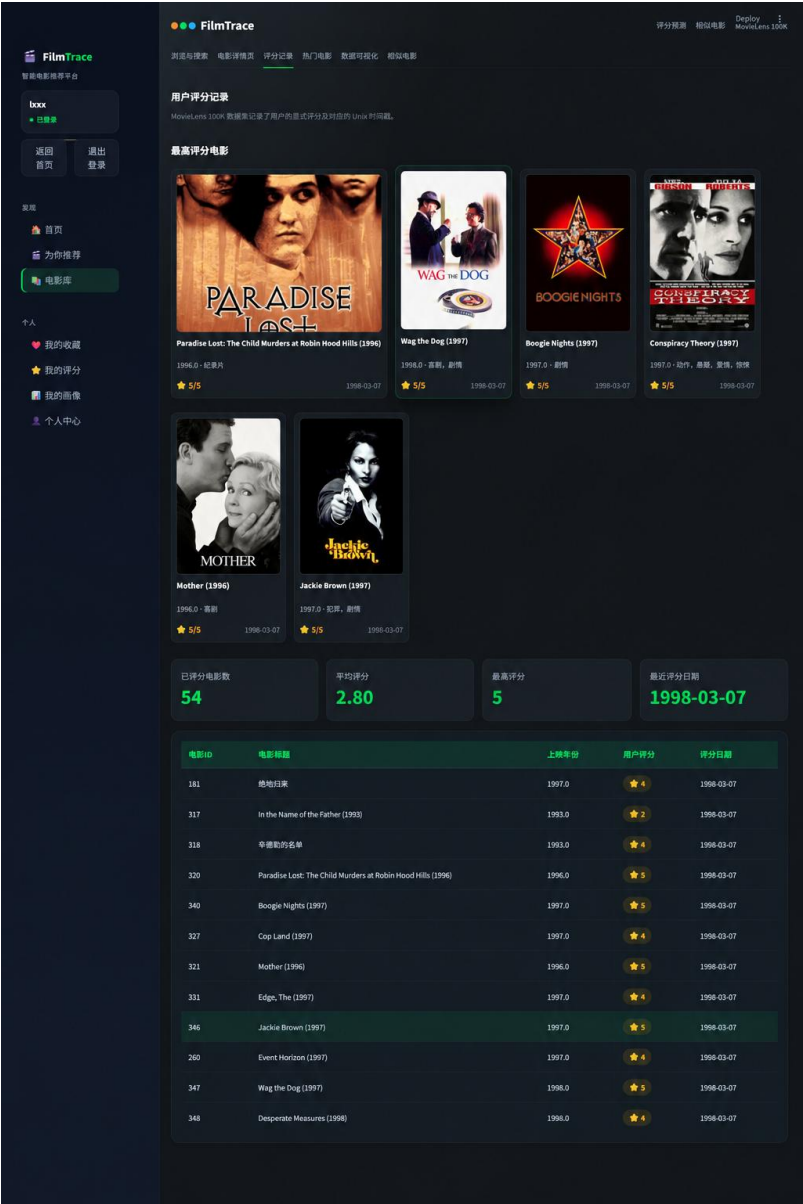


图 16 用户评分记录与偏好分析界面

图 17 所示为“我的评价”界面，展示用户撰写的文字评价。评价写入 ratings 表的 review 字段，与评分共同构成用户对电影的显式反馈。

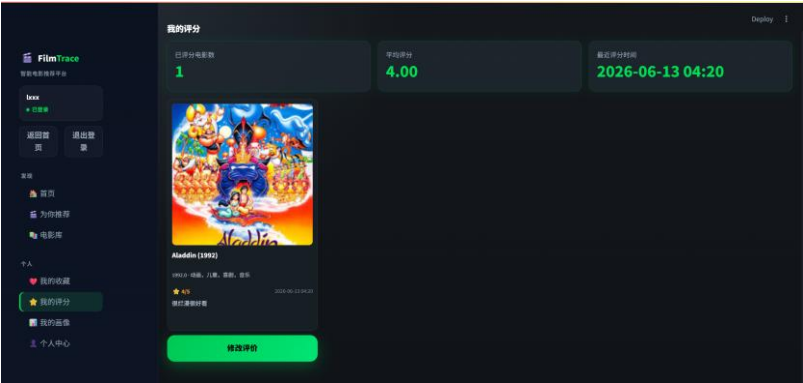


图 17 我的评价界面

图 18 所示为收藏管理界面，展示并管理用户收藏的电影。收藏数据存于 favorites 表，收藏行为亦可作为兴趣建模与推荐的辅助信号。

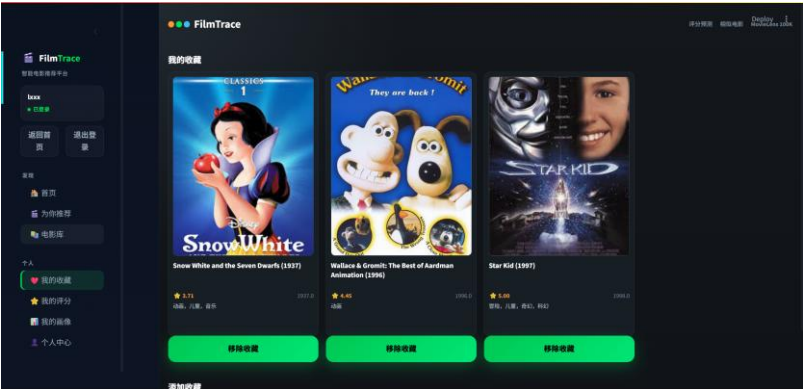


图 18 用户收藏管理界面

图 19 所示为个人中心界面，集中展示账号信息与个人数据入口，便于用户管理账号与偏好设置。

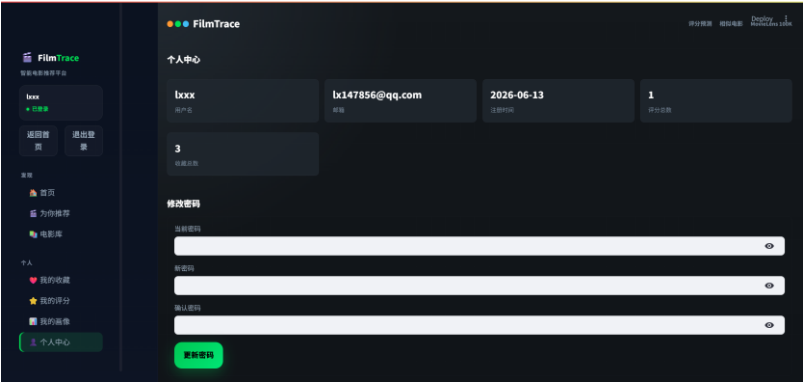
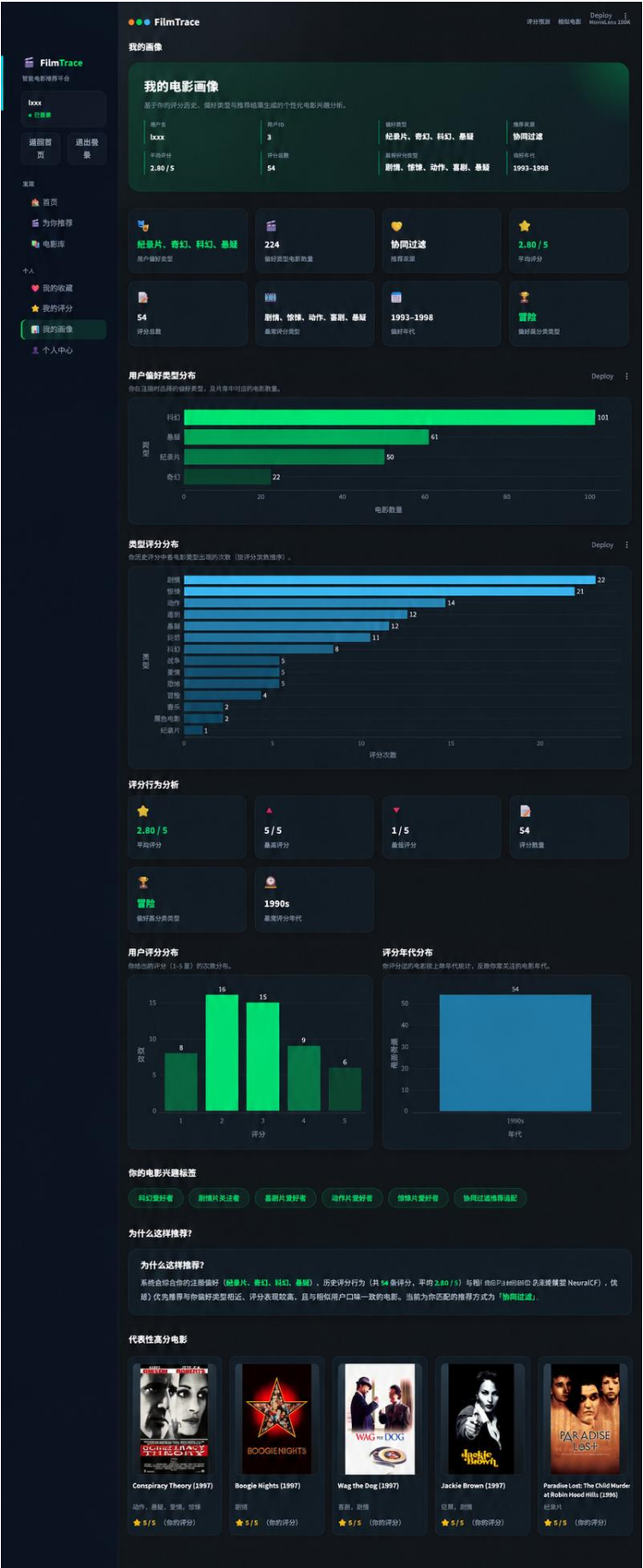


图 19 用户个人中心界面

图 20 所示为用户电影画像分析界面。系统基于评分与收藏行为，统计偏好类型、年代分布与评分习惯，生成个性化画像并写入 user_profiles 表（top_genres、year_range、preferred_min_rating 等），将分散的行为数据转化为直观画像，提升系统可解释性。



7.6 数据可视化

图 21 所示为用户端数据可视化分析界面，借助 Plotly/Matplotlib 将评分分布、类型偏好、推荐结果分布等以交互图表呈现，帮助用户从整体上理解自身观影口味与平台数据特征，是系统可解释性的重要体现。



图 21 用户端数据可视化分析界面

7.7 管理员后台功能实现

图 22 所示为管理员后台首页。管理员登录后进入独立后台，可访问电影库管理、用户管理、评分记录、算法配置、模型评估、系统统计等功能；所有写操作经 `src/db.py` 执行并记录至 `admin_audit_log`，实现权限隔离与操作留痕。

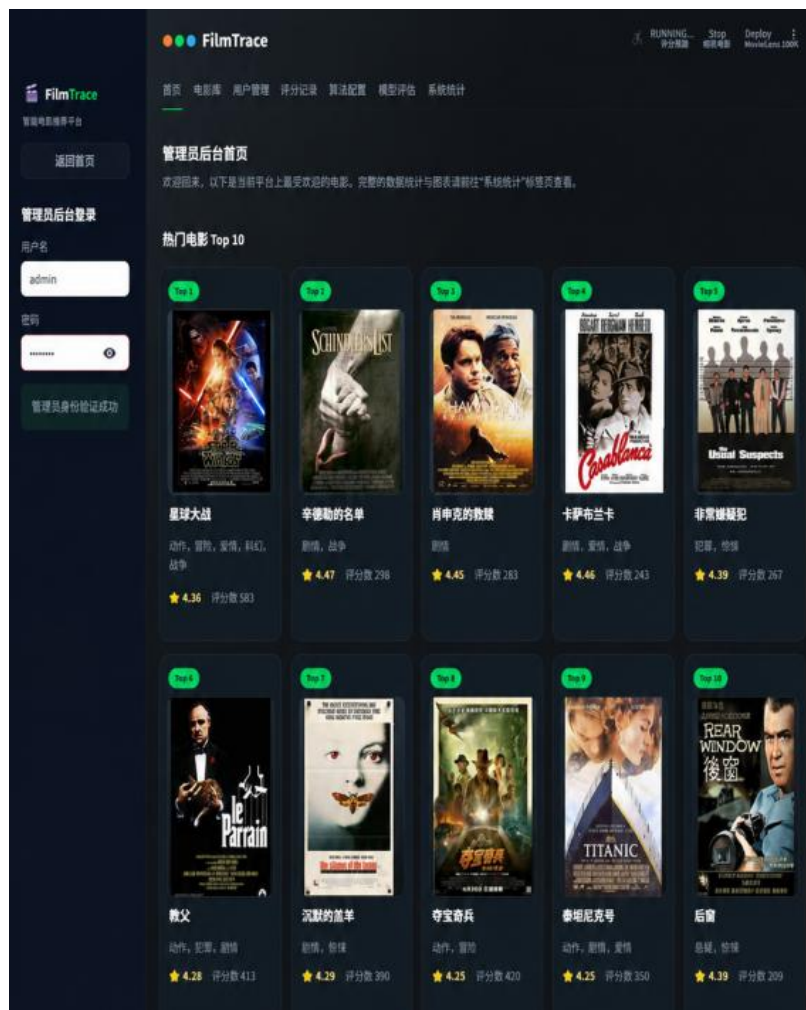


图 22 管理员后台首页

图 23 所示为管理员电影数据管理界面，以海报卡片展示并支持对 movies 表中电影信息的查询与管理，便于维护电影库内容。

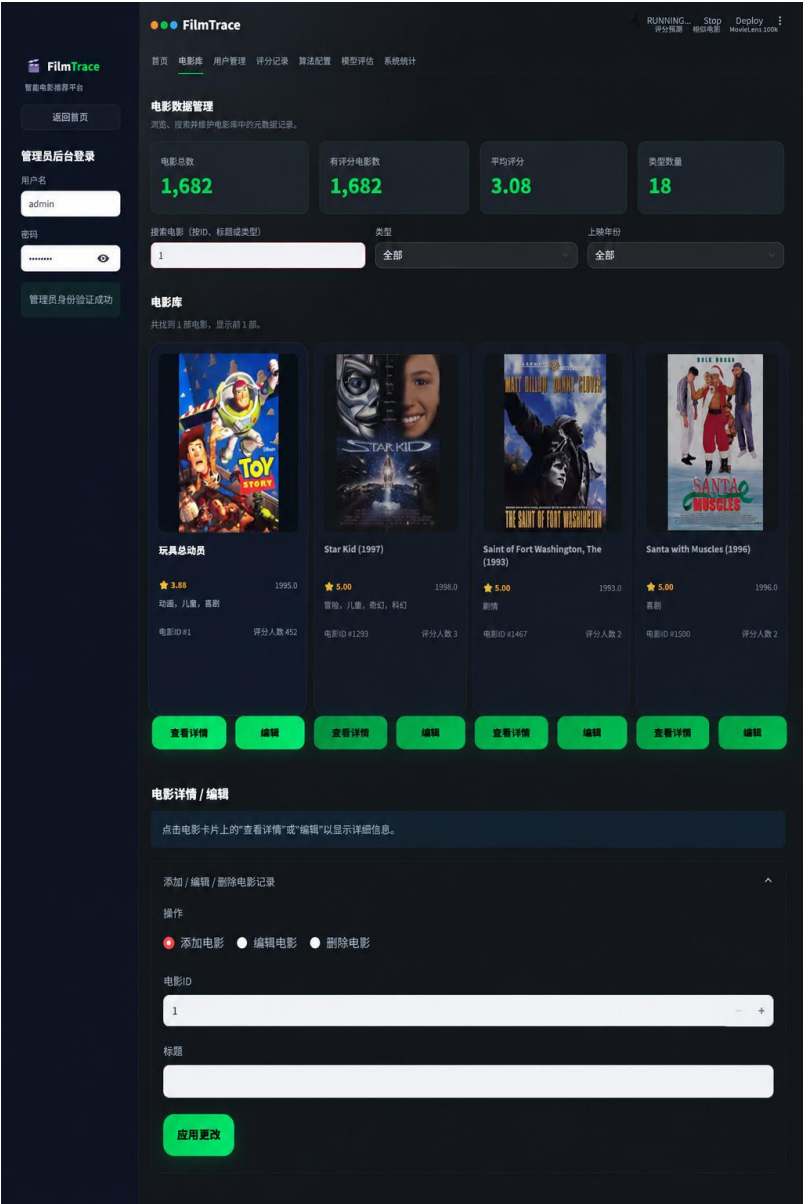


图 23 管理员电影数据管理界面

图 24 所示为管理员用户管理界面，统计各用户的评分次数、平均评分、活跃度并可推断其偏好类型，便于运营方掌握用户规模与活跃情况。

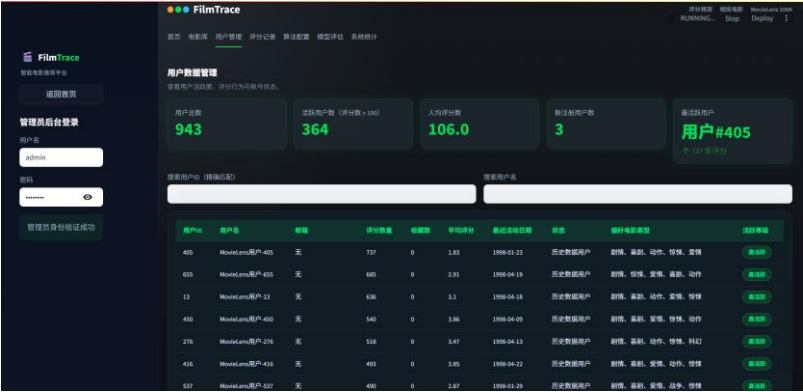


图 24 管理员用户管理界面

图 25 所示为管理员评分记录管理界面，集中管理 ratings 表中的用户-电影交互数据，是协同过滤的核心数据来源，也为数据质量核查提供入口。

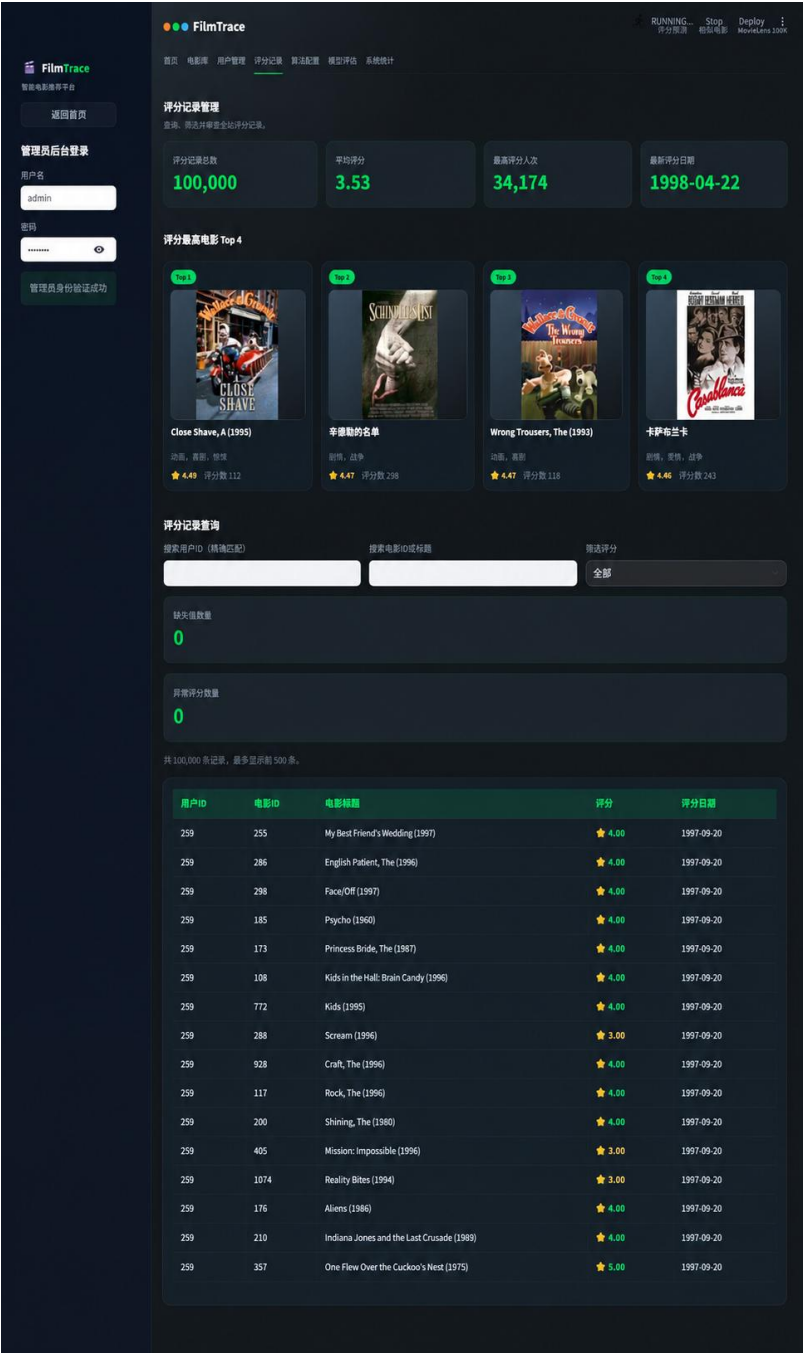


图 25 管理员评分记录管理界面

图 26 所示为推荐算法配置界面，管理员可选择推荐算法（UserCF/ItemCF/SVD/Hybrid/NeuralCF）并调整近邻数 K、SVD 隐因子数等关键参数，实现对推荐策略的灵活配置。

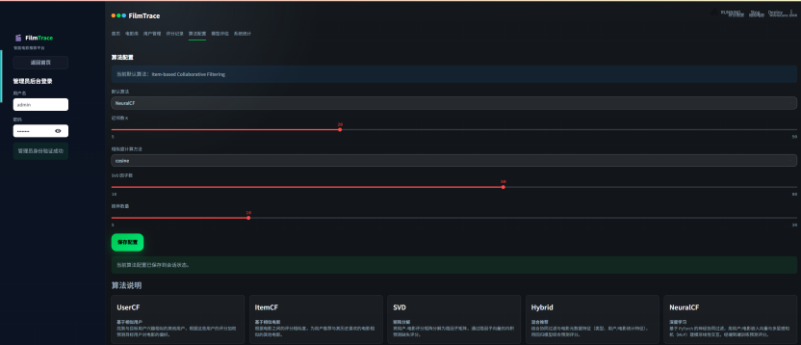


图 26 推荐算法配置界面

图 27 所示为推荐模型评估界面，管理员可在此查看各模型在 MAE、RMSE 及 Precision@10/Recall@10/HitRate@10/NDCG@10 上的评估结果与对比图表，将第 8、9 章的离线评估结论以可视化方式呈现，支撑算法选型。

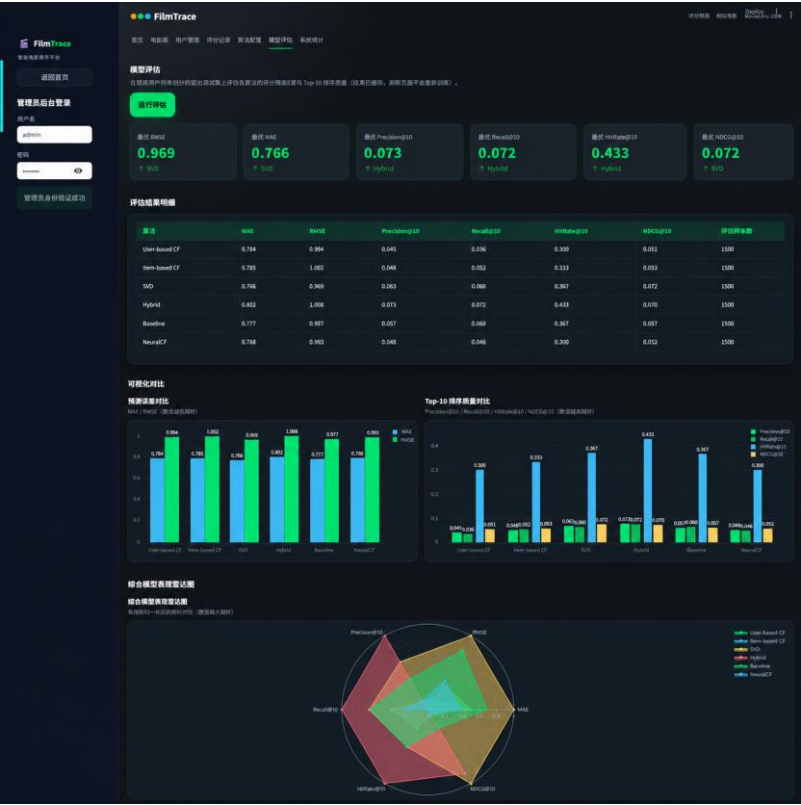


图 27 推荐模型评估界面

图 28 所示为系统统计与数据分析界面，汇总用户、电影、评分的总体统计，并呈现类型分布、评分分布与热门电影分析，为运营方提供系统运行状况的全局视图。



图 28 系统统计与数据分析界面

8 实验设计与模型评估

8.1 评估协议与指标

实验在按用户时间序列划分的训练/测试集上进行：所有相似度矩阵、均值、偏置项、混合特征与隐因子均仅在训练集上拟合，RMSE/MAE 仅在留出测试评分上计算，以杜绝信息泄漏。Top-N 评估针对每位用户的留出评分，采用全目录候选排序，默认以测试评分 ≥ 4.0 为相关判定，K 取 10，参与排序评估的用户数为 60。各指标定义见表 6。

表 6 评估指标定义

指标	定义	评估角度
RMSE	均方根误差，对较大误差惩罚更重	评分预测精度
MAE	平均绝对误差	评分预测精度
Precision@K	Top-K 列表中相关物品（评分 $\geq$ 阈值）的比例	排序质量
Recall@K	相关物品被召回进 Top-K 的比例	排序质量
HitRate@K	Top-K 中至少命中一个相关物品的用户比例	命中能力
NDCG@K	考虑相关物品排序位置的归一化折损累积增益	排序质量
Catalog Coverage	推荐结果覆盖的电影占电影库的比例	多样性/覆盖

8.2 实验数

主要实验参数见表 7，所有随机过程固定 `random_state=42`。

表 7 主要实验参数

参数	取值	说明
<code>random_state</code>	42	全局随机种子
<code>test_size</code>	0.2	测试集比例（按用户时序划分）
<code>K_NEIGHBORS</code>	20	UserCF/ItemCF 近邻数（敏感性范围 5-50）
<code>SVD_N_COMPONENTS</code>	50	SVD 隐因子数
偏置正则系数 <code>reg</code>	1.0	正则化偏置基线最优值
NeuralCF	<code>emb=32,</code> <code>MLP(64,32),</code> <code>epochs=20</code>	AdamW(<code>lr=1e-3</code> , <code>wd=1e-5</code>), <code>batch=512</code> , MSE 损失
相关性阈值 / K	4.0 / 10	Top-N 相关判定与列表长度

8.3 评估结果

图 29、图 30 分别给出各模型在评分预测（RMSE/MAE）与 Top-N 排序（Precision@10/Recall@10/NDCG@10）上的对比；表 8 汇总关键结果，数据来源为执行后的 notebook（`notebooks/01_movielens_recommendation_executed.ipynb`），是本项目结果的唯一真值源。

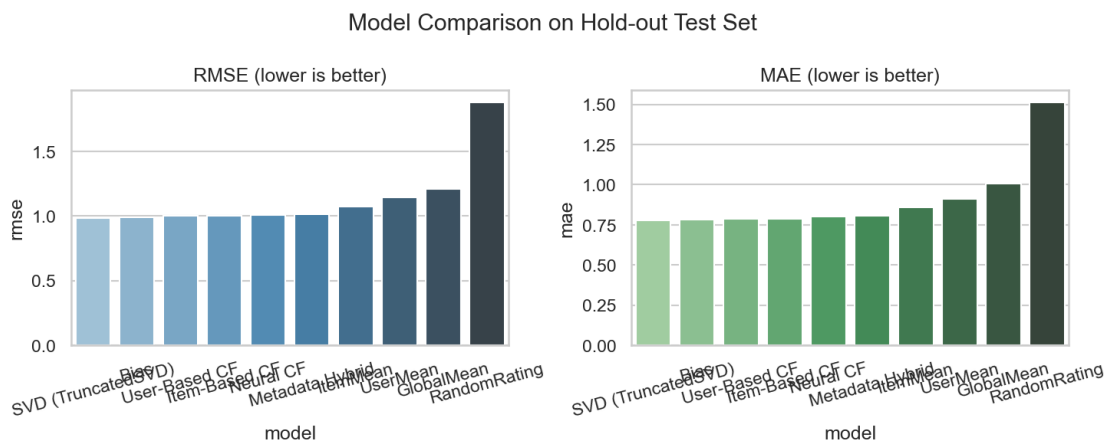


图 29 各模型评分预测误差（RMSE/MAE）对比

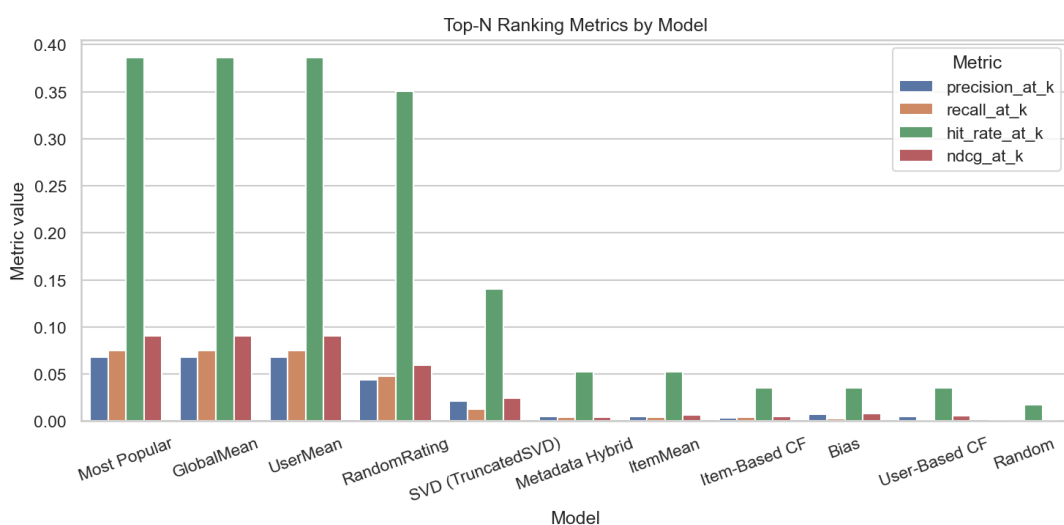


图 30 各模型 Top-N 推荐指标对比

表 8 关键评估结果汇总

指标	最优模型	数值
RMSE	SVD（偏置残差 TruncatedSVD）	0.9826
MAE	SVD（偏置残差 TruncatedSVD）	0.7763
RMSE / MAE（深度模型）	NeuralCF（PyTorch）	1.0080 / 0.8003
Precision@10	Most Popular / GlobalMean / UserMean	0.0684
Recall@10	Most Popular / GlobalMean / UserMean	0.0752
NDCG@10	Most Popular / GlobalMean / UserMean	0.0908
Catalog Coverage（随机基线）	Random	0.3014
Catalog Coverage（最优非随机）	User-Based CF	0.0916

8.4 相关性阈值敏感性

Top-N 指标对相关性阈值较敏感，故对阈值 3.0/3.5/4.0 分别评估，结果见表 9。

表 9Top-N 相关性阈值敏感性

阈值	最优 Precision@10	最优 Recall@10	最优 NDCG@10
3.0	0.0883	0.0691	0.1113
3.5	0.0684	0.0752	0.0908
4.0	0.0684	0.0752	0.0908

三档阈值下取得最优 Top-N 指标的始终为 Most Popular/GlobalMean/UserMean 等流行度类基线，说明该现象在不同阈值下稳定存在，并非阈值选择所致。

8.5 超参数敏感性

图 301 给出 UserCF/ItemCF 在近邻数 K（5–50）下的 RMSE 趋势，图 32 给出偏置基线在不同正则化系数下的 RMSE 趋势（据此确定 $\text{reg}=1.0$ ）。由于传统模型不依赖梯度迭代，故以此类敏感性曲线替代深度学习的训练损失曲线。

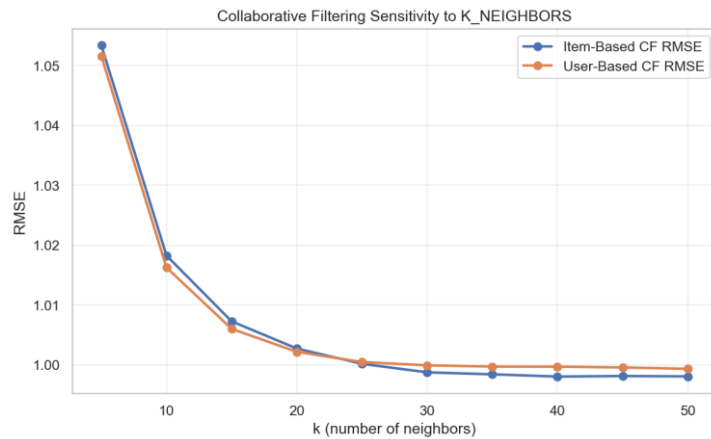


图 31UserCF/ItemCF 近邻数 K 对 RMSE 的影响

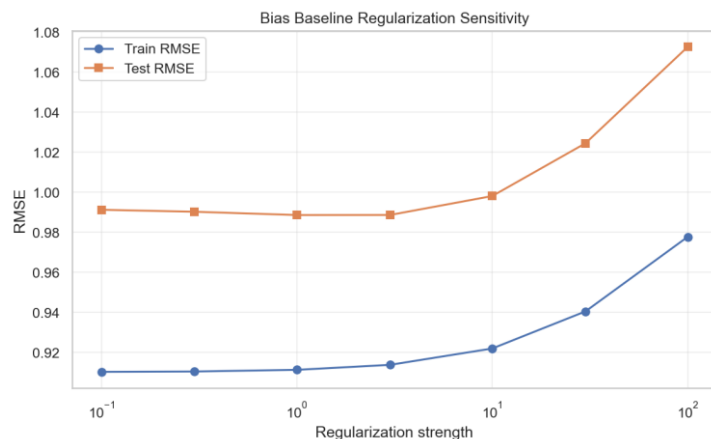


图 32 偏置基线正则化系数对 RMSE 的影响

8.6NeuralCF 训练过程可视化

NeuralCF 通过小批量随机梯度下降端到端训练，每个 epoch 的平均训练损失

(MSE) 被记录并由 ExperimentLogger 写入 experiments/neuralcf/metrics.json，训练损失曲线如图 33。模型在 80,000 条训练评分上训练 20 个 epoch。

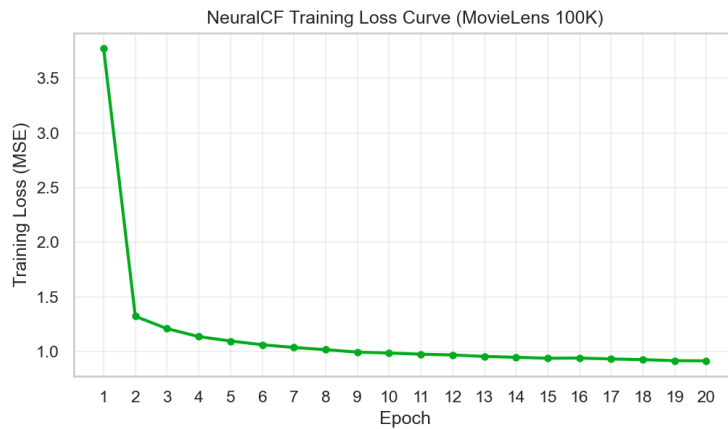


图 33 NeuralCF 训练损失 (MSE) 随 epoch 变化曲线

由图 33 可见，训练损失在首个 epoch 后由约 3.77 迅速降至约 1.32，随后平稳收敛，至第 20 个 epoch 降至约 0.91，曲线无明显震荡或回升，表明训练稳定、未见明显过拟合。NeuralCF 在测试集上取得 RMSE=1.0080、MAE=0.8003，与传统协同过滤同量级，但略逊于偏置残差 SVD (RMSE=0.9826)，这与已有研究中“中小规模稀疏显式评分数据上结构简单的神经模型未必优于精心设计的矩阵分解”的观察一致。

9 结果分析

9.1 定量结果分析

从评分预测看，偏置残差 SVD 以 RMSE=0.9826、MAE=0.7763 优于包括 NeuralCF 在内的其余模型，验证了“先拟合偏置、再分解残差”这一两阶段思路在稀疏矩阵上的有效性，也说明在评分预测这一回归任务上，矩阵分解能比纯邻域方法与简单神经网络充分地利用潜在交互信号。

从 Top-N 排序看，结果出现值得深究的反直觉现象：在全目录离线评估下，Most Popular、GlobalMean、UserMean 等流行度类基线在 Precision@10、Recall@10、NDCG@10 上反超 UserCF、ItemCF、SVD、Hybrid、NeuralCF 等个性化模型，且该现象在 3.0/3.5/4.0 三档阈值下一致出现。

9.2 错误案例与原因分析

结合实验记录与相关研究，上述现象主要由评估协议与数据特性共同造成：

- 评估协议层面：离线 Top-N 将“未评分”视为“未知”而非真实负样本，全目录排序中每部未观看电影都参与竞争 Top-10，但仅少量留出评分能计为正样本，这会系统性压低 Precision/Recall，并使流行度基线因“热门即高命中概率”而占优（与 Cremonesi 等人结论一致[5]）；
- 数据稀疏层面：评分矩阵稀疏度达 93.7%，用户/物品间可用于估计可靠邻域的共同评分稀少，导致邻域方法的排序信号不足；
- 评估规模层面：参与 Top-N 评估的用户为 60 名、每人留出正样本有限，进一步放大了排序指标的波动。

因此本报告将该结果解读为“在此离线协议下个性化排序器尚未提取出足够排序信号”的警示，而非个性化算法完全失效；更贴近生产实践的做法是将流行度先验、偏置项、隐因子与重排序相结合。

9.3 覆盖率与多样性

图 334 给出各模型多样性与新颖性对比。结合覆盖率可见：**Random** 基线覆盖率最高（0.3014）但质量最低，说明“覆盖率高不等于质量好”；非随机模型中 **User-Based CF** 覆盖率最高（0.0916），推荐分布相对更广，但与 **Random** 仍有明显差距。这清晰展示了推荐系统中“精度—覆盖率/多样性”的典型权衡。

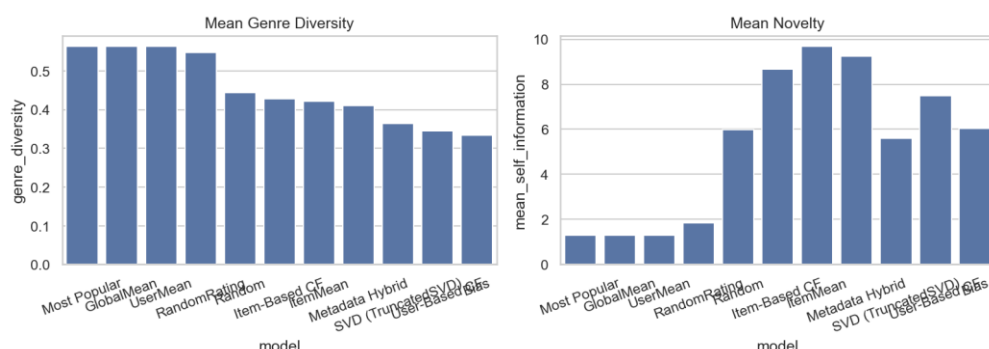


图 34 各模型多样性与新颖性对比

9.4 冷启动与定性结果

针对新用户冷启动，系统通过注册偏好类型结合元数据混合模型生成初始推荐，在缺乏历史评分时仍能给出与偏好类型相关的结果。从定性看（参见 7.4 节），系统能结合历史评分与偏好给出带预测评分和推荐理由的列表，推荐结果与用户画像（7.5 节）在偏好类型上保持一致，体现了较好的可解释性。此外，本地海报匹配在迭代中不断完善：通过片名规范化与占位图兜底，保证了即便个别电影缺失海报，界面仍能稳定渲

染。

9.5 局限性

- MovieLens 100K 规模较小、年代较早，结论不宜直接外推到大规模实时业务；
- 全目录离线 Top-N 协议将“未评分”视为“不相关”，可能低估个性化模型的真实排序能力，且与采样负例基准不可直接比较；
- 纯协同过滤对新用户/新电影的冷启动仅得到缓解而非根本解决；
- 所有结论均基于离线数据，未引入在线 A/B 测试与真实点击日志。

10 项目测试与运行说明

为保证可复现性，本节给出从环境准备到系统运行的完整步骤。

10.1 运行环境与依赖

项目基于 Python 实现，主要依赖 pandas、NumPy、scikit-learn、Streamlit、Plotly/Matplotlib，深度模型 NeuralCF 依赖 PyTorch。项目同时提供 environment.yml 与 requirements.txt，列明全部依赖及版本，可在 conda 或 pip 环境下复现。配置项集中于 src/config.py，并提供 configs/config.yaml 供脚本读取同一套超参数。

10.2 数据与目录

- 原始数据由 src/data_loader.py 加载，本地海报由 src/posters.py 匹配；
- 图表输出位于 figures/，系统截图位于 report/screenshot/，实验日志位于 experiments/；
- SQLite 数据库 app.db 在系统首次运行时自动创建并落库。

10.3 运行步骤

(1) 安装依赖（任选其一）：

```
conda env create -f environment.yml
pip install -r requirements.txt
```

(2) 复现数据分析与模型评估：

```
jupyter nbconvert --to notebook --execute
notebooks/01_movielens_recommendation.ipynb
```

(3) 启动 Web 应用：

```
streamlit run app.py
```


(4) 运行单元测试：

```
pytest -q
```

源代码托管于 GitHub: [https://github.com/xuanlv361-ctrl/MovieLens-](https://github.com/xuanlv361-ctrl/MovieLens-Recommendation-System)

Recommendation-System, 包含完整代码、图表、截图、实验日志与报告材料。

10.4 可复现性

项目所有随机过程统一使用 `src/reproducibility.py` 中的 `set_seed(42)`（同时设定 Python、NumPy、PyTorch 及 cuDNN 的确定性），预测评分裁剪至 [1,5]；代码遵循 PEP8（经 ruff 校验通过），关键函数/类均含 docstring，未使用硬编码路径；notebooks/01_movielens_recommendation_executed.ipynb 为已执行副本，可作为结果复现依据。

11 团队分工

11.1 分工说明

本项目由 3 名成员协作完成，分工与工作量占比如表 10。

表 10 团队分工与工作量占比

负责人	工作模块	主要内容
吕璇	数据处理、推荐算法、系统开发、模型评估、数据可视化与报告整理	负责 MovieLens 100K 数据集的读取、清洗与预处理，构建用户-电影评分矩阵，完成电影类型解析与本地海报资源匹配；实现并对比 9 类传统推荐模型（全局/用户/物品均值、正则化偏置、随机、最受欢迎等基线，以及 UserCF、ItemCF、基于偏置残差的 SVD 矩阵分解、元数据混合模型），并实现基于 PyTorch 的 NeuralCF 神经协同过滤深度学习模型；完成 MAE、RMSE、Precision@K、Recall@K、HitRate@K、NDCG@K 等评估指标的计算、相关性阈值与超参数敏感性分析及结果分析；基于 Streamlit 开发用户端与管理员端完整系统，包括欢迎页、用户注册登录与偏好选择、电影库与电影详情、个性化推荐、相似电影、评分与评价、收藏、用户画像、数据可视化，以及管理员后台的电影管理、用户管理、评分记录、算法配置、模型评估与系统统计等模块；设计并实现基于 SQLite 的数据持久化层（movies、users、ratings、accounts、favorites、user_preferences、user_profiles、admin_audit_log 共 8 张表）；开发数据可视化模块，绘制评分分布、类型分布、各模型指标对比、

负责人	工作模块	主要内容
		超参数敏感性与 NeuralCF 训练损失曲线等图表；编写单元测试并完成代码规范化（PEP8、docstring、固定随机种子与可复现性配置）；整理系统截图、ER 图、系统架构图与 Git 提交记录，并完成项目报告的撰写与排版。

11.2 项目反思

在项目推进过程中，遇到的主要困难有三：其一，MovieLens 100K 评分矩阵稀疏度高达 93.7%，导致 UserCF/ItemCF 在共同评分稀少时相似度估计不稳定。我们通过在偏置基线中引入 L2 正则化、在 SVD 中采用“偏置 + 残差分解”的两阶段策略予以缓解。其二，离线 Top-N 评估中个性化模型未能超过流行度基线，这一反直觉结果一度令人困惑；我们没有回避，而是查阅文献后定位到“全目录排序协议将未评分视为不相关”是主要成因，并在报告中如实呈现与分析。其三，本地海报文件名与 MovieLens 片名存在格式差异导致匹配失败，我们通过片名规范化与占位图兜底解决了界面渲染异常的问题。

回顾整个项目，较好地完成了从数据处理、多模型对比到系统落地的完整闭环，9 类传统模型与 NeuralCF 的统一评估、以及对“反直觉结果”的理性分析是主要亮点。不足之处在于：受课程时间与数据规模限制，未引入采样负例评估等更前沿的评估协议；系统的算法配置与模型评估仍停留在离线层面，尚未与在线推荐效果打通。未来若继续推进，团队将按第 13 章提出的方向，在评估协议、深度模型与在线能力等方面逐步改进，并进一步加强成员间在代码评审与文档协作上的规范化。

12 Git 版本控制与项目管理

本项目全程使用 Git 进行版本控制并托管于 GitHub。提交历史覆盖从数据处理、算法实现、系统开发到报告撰写的完整周期，反映了项目的迭代过程。图 35 为 Git 提交记录截图，提交主页见 <https://github.com/xuanlv361-ctrl/MovieLens-Recommendation-System/commits/main>。

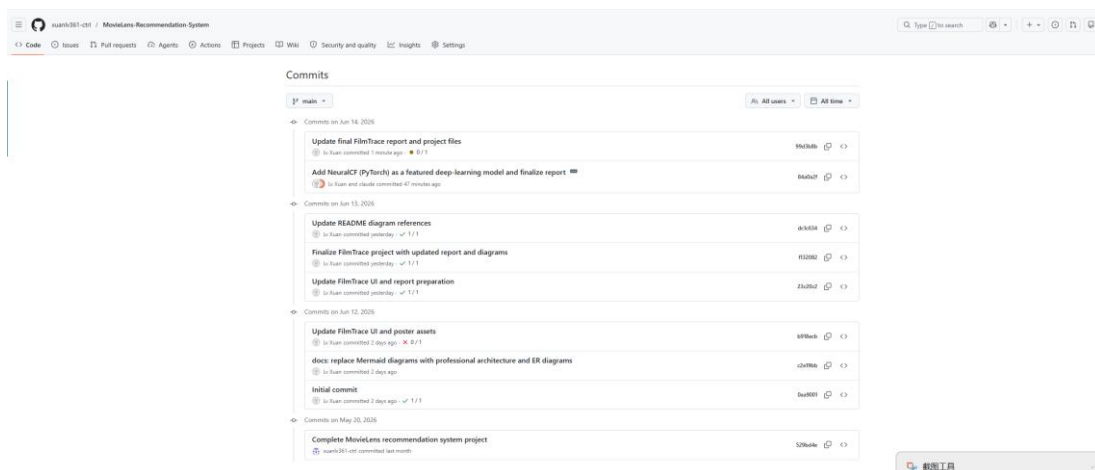


图 35GitHub 提交记录截图

最终仓库包含完整源代码、依赖文件、图表、系统截图、实验日志与报告材料。借助 `git log` 可统计各成员的提交次数与代码改动量，作为团队分工中工作量占比的客观参考依据。

13 结论与展望

本项目以 MovieLens 100K 为基础，完成了从数据清洗、特征预处理、训练/测试划分，到 11 种推荐模型实现与统一评估的完整流程，并据此构建了基于 Streamlit 与 SQLite 的 FilmTrace 推荐系统，实现了用户端与管理端的功能闭环。主要发现为：偏置残差 SVD 在评分预测上取得最优 RMSE (0.9826) 与 MAE (0.7763)，优于 NeuralCF 等模型；在全目录离线 Top-N 协议下，流行度类基线在排序指标上反超个性化模型，经分析主要源于评估协议本身与数据高度稀疏；User-Based CF 在非随机模型中覆盖率最高 (0.0916)，体现精度与覆盖率的权衡。

本项目的局限在于：数据集规模较小且年代较早；离线评估协议可能低估个性化模型的真实排序能力；冷启动问题仅得到缓解；系统尚不具备在线学习与实时反馈能力。

未来可在以下方向继续改进：在更大规模的 MovieLens 1M/10M 数据集上验证算法与系统的可扩展性；改进 Top-N 评估协议（引入采样负例或限定候选集）以更公平地比较个性化模型与流行度基线；引入更复杂的深度推荐模型并纳入统一评估；在 Web 端引入用户实时行为日志与在线更新机制，并探索在线部署；进一步丰富用户画像建模与海报/元数据质量，提升推荐的可解释性与冷启动表现。

参考文献

- [1] Harper F M, Konstan J A. The MovieLens Datasets: History and Context[J]. ACM Transactions on Interactive Intelligent Systems, 2015, 5(4): 1-19.
- [2] Sarwar B, Karypis G, Konstan J, et al. Item-Based Collaborative Filtering Recommendation Algorithms[C]//Proceedings of the 10th International Conference on World Wide Web (WWW). 2001: 285-295.
- [3] Koren Y, Bell R, Volinsky C. Matrix Factorization Techniques for Recommender Systems[J]. Computer, 2009, 42(8): 30-37.
- [4] Herlocker J L, Konstan J A, Terveen L G, et al. Evaluating Collaborative Filtering Recommender Systems[J]. ACM Transactions on Information Systems, 2004, 22(1): 5-53.
- [5] Cremonesi P, Koren Y, Turrin R. Performance of Recommender Algorithms on Top-N Recommendation Tasks[C]//Proceedings of the 4th ACM Conference on Recommender Systems (RecSys). 2010: 39-46.
- [6] Burke R. Hybrid Recommender Systems: Survey and Experiments[J]. User Modeling and User-Adapted Interaction, 2002, 12(4): 331-370.
- [7] He X, Liao L, Zhang H, et al. Neural Collaborative Filtering[C]//Proceedings of the 26th International Conference on World Wide Web (WWW). 2017: 173-182.
- [8] Ricci F, Rokach L, Shapira B. Recommender Systems Handbook[M]. 2nd ed. Boston: Springer, 2015.
- [9] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine Learning in Python[J]. Journal of Machine Learning Research, 2011, 12: 2825-2830.
- [10] McKinney W. Data Structures for Statistical Computing in Python[C]//Proceedings of the 9th Python in Science Conference (SciPy). 2010: 56-61.
- [11] Paszke A, Gross S, Massa F, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library[C]//Advances in Neural Information Processing Systems (NeurIPS). 2019: 8024-8035.
- [12] Streamlit Inc. Streamlit Documentation[EB/OL]. (2024)[2026-06-14]. <https://docs.streamlit.io>.
- [13] Gheewala S, Xu S, Yeom S. In-depth Survey: Deep Learning in Recommender Systems—Exploring Prediction and Ranking Models, Datasets, Feature Analysis, and Emerging Trends[J]. Neural Computing and Applications, 2025, 37(17): 10875-10947.
- [14] Chen X, Yao L, McAuley J, et al. Deep Reinforcement Learning in Recommender Systems: A Survey and New Perspectives[J]. Knowledge-Based Systems, 2023, 264: 110335.

[15] Liu Y, Lin T, Ye X. Federated Recommender Systems Based on Deep Learning: The Experimental Comparisons of Deep Learning Algorithms and Federated Learning Aggregation Strategies[J]. Expert Systems with Applications, 2024, 239: 122440.

[16] Talha M M, Khan H U, Iqbal S, et al. Deep Learning in News Recommender Systems: A Comprehensive Survey, Challenges and Future Trends[J]. Neurocomputing, 2023, 562: 126881.

附录

附录 A 实验记录表（含未达预期的尝试）

下表汇总主要实验的关键超参数、结果与未达预期的尝试，结果均来自执行后的 notebook。

实验	关键超参数	结果	结论 / 是否采用
偏置正则系数搜索	reg ∈ {0.1,...,10}	reg=1.0 时 RMSE 最低	采用 reg=1.0
UserCF/ItemCF 近邻数搜索	K ∈ {5,...,50}	K 过小方差大，K=20 趋稳	采用 K=20
SVD 隐因子数搜索	n_components ∈ {10,...,80}	n=50 后趋稳，过大易过拟合	采用 n=50
个性化模型用于 Top-N	全目录, rating≥4.0, K=10	排序指标低于流行度基线	未达预期；归因于评估协议（见 9.2）
NeuralCF 深度模型	emb=32, MLP(64,32), 20 epoch	RMSE 1.0080，略逊于 SVD	已实现但未超越 SVD（见 8.6）
随机/全局时间划分	随机划分 / 全局时序	存在泄漏或不贴近场景	仅作参考，未用作主实验

附录 B 可复现性与实验日志

所有随机过程统一使用 set_seed(42)，预测评分裁剪至 [1,5]。NeuralCF 训练时由 ExperimentLogger 将运行配置与每轮训练损失分别写入 experiments/neuralcf/config.json 与 metrics.json，连同已执行的 notebook 副本共同作为结果复现依据。